

03_data-overview

July 6, 2026

```
[1]: # %%
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from consts import *
import numpy as np
from itertools import combinations
from scipy.stats import linregress
from scipy.stats import spearmanr
from scipy.stats import pearsonr
import json

plt.rcParams.update({"font.size":9})
plt.rcParams.update({"figure.figsize":(16/2.54, 9/2.54)})
sns.set_style("ticks")
sns.set_context("paper")
```

1 Variable Overview

1.0.1 Load Data

```
[2]: df_orig = pd.read_csv("processed_data/2026-05-13_allBilendiData.csv")

[3]: # %%
df_p = pd.read_csv("processed_data/2026-06-19_data_processed_participant.csv")
print("Full Size of participant data: ", df_p.wave.value_counts().to_dict())

inds_bothwaves = df_p["id"].value_counts().reset_index().
    ↳query("count==2")["id"].tolist()
df_p_bothwaves = pd.read_csv("processed_data/
    ↳2026-06-19_data_processed_participant_pivot.csv").query(f"id in_
    ↳{inds_bothwaves}")
print("Full Size of across-wave data: ", len(df_p_bothwaves))
```

Full Size of participant data: {1: 1170, 2: 961}

Full Size of across-wave data: 903

```
[4]: # %%
df_diff = pd.read_csv("processed_data/2026-06-19_data_processed_differences.
↳csv")
# df_diff = df_diff.loc[((df_diff["id"].isin(ids_w1)) & (df_diff["wave"]==1)) |
↳((df_diff["id"].isin(ids_w2)) & (df_diff["wave"]==2))]
print("Full Size of pairwise data (in wave 1 and wave 2): ", df_diff["wave"].
↳value_counts().to_dict())
print(df_diff.shape)
print(len(df_diff.loc[df_diff.wave==1,"id"].unique()))
print(len(df_diff.loc[df_diff.wave==2,"id"].unique()))
# print(df_diff.columns)

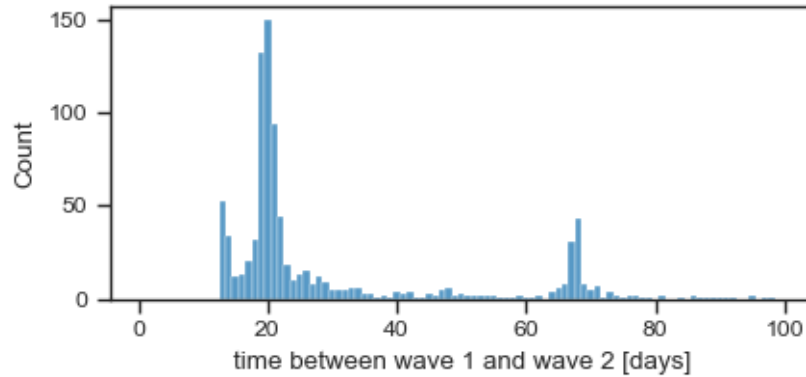
cols_to_keep = ["id", "wave"] + [f"std_socialCircle_ops_{q}" for q in
↳questions_sc]
cols_to_keep += ["average_pixel_dist", "average_pixel_dist_parties"]
df_diff = df_diff.merge(df_p[cols_to_keep], on=["id", "wave"], how="left")
df_diff["rel_pixel_dist"] = df_diff["pixel_dist"]/df_diff["average_pixel_dist"]
df_diff["rel_pixel_distP"] = df_diff["pixel_dist"]/
↳df_diff["average_pixel_dist_parties"]
```

Full Size of pairwise data (in wave 1 and wave 2): {1: 150450, 2: 125463}
 (275913, 35)
 1170
 961

2 Time Variables

```
[5]: # %%
plt.figure(figsize=(12/2.54, 5/2.54))
sns.histplot((pd.to_datetime(df_p_bothwaves["wave2_t_completed"]) - pd.
↳to_datetime(df_p_bothwaves['wave1_t_completed'])).dt.days, bins=np.arange(0.
↳5,100.5))
plt.xlabel("time between wave 1 and wave 2 [days]" )
```

[5]: Text(0.5, 0, 'time between wave 1 and wave 2 [days]')



```
[6]: # %%
fig, ax = plt.subplots(1,1, sharex=False, figsize=(3,2))
t = 'time_total'
vmax = np.percentile(df_p.assign(t_div_60=df_p[t] / 60)['t_div_60'].values, 95)
    ↳ * 3
df_p = df_p.copy()
sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave", bins=
    ↳ np.arange(0,vmax,3), alpha=0.5, palette=cmapWave, hue_order=[2,1])
ax.set_title(t)
ax.text(0.95,0.95,
        "\n".join([f"{np.sum(df_p.loc[df_p.wave==w, t].values / 60 < vmax)/df_p.
    ↳ loc[df_p.wave==w, t].count()*100:.1f}% with <{vmax:.1f}min (wave {w})" for w
    ↳ in [1,2]]),
        ha="right", va="top", transform=ax.transAxes, fontsize=7)
ax.set_xlabel("time in min")
fig.tight_layout()

print(f"median ({t}): {df_p[t].div(60).median():.2f} (25%-perc: {df_p[t].
    ↳ div(60).describe()['25%']:.2f}, 75%-perc: {df_p[t].div(60).describe()['75%']:
    ↳ .2f}))")

times = ['time_trainingGame', 'time_training', 'time_spam', 'time_spam18dots',
    ↳ 'time_pairwise', 'time_pairwise18pairs']
fig, axs = plt.subplots(2,3, sharex=False, figsize=(7,3))
for ax, t in zip(axs.flatten(), times):
    vmax = np.percentile(df_p[t].dropna().values, 95) * 3 / 60
    df_p = df_p.copy()
    sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave",
    ↳ bins = np.arange(0,vmax,0.5), alpha=0.5, ax=ax, legend=False,
    ↳ palette=cmapWave, hue_order=[2,1])
    ax.set_title(t)
    ax.text(0.95,0.95,
```

```

        "\n".join([f"{np.sum(df_p.loc[df_p.wave==w, t].values / 60 < vmax)/
↳df_p.loc[df_p.wave==w, t].count()*100:.1f}% with <{vmax:.1f}min (wave {w})"
↳for w in [1,2]]),
        ha="right", va="top", transform=ax.transAxes, fontsize=7)
    ax.set_xlabel("time in min")
fig.tight_layout()

display(df_p[times].div(60).describe())
display(df_p[times].describe())

# for t in [4,6]:
#     descr = df_p[time_cols(1)[t]].div(60).describe(percentiles=[0.05,0.25,0.
↳5,0.75,0.95])
#     print(f"median ({time_cols(1)[t]}): {descr["50%"]:.2f} (25%-perc:
↳{descr["25%"]:.2f}, 75%-perc: {descr["75%"]:.2f}), (5%-perc: {descr["5%"]:.
↳2f}, 95%-perc: {descr["95%"]:.2f})")
#     descr = df_p[time_cols(2)[t]].div(60).describe(percentiles=[0.05,0.25,0.
↳5,0.75,0.95])
#     print(f"median ({time_cols(2)[t]}): {descr["50%"]:.2f} (25%-perc:
↳{descr["25%"]:.2f}, 75%-perc: {descr["75%"]:.2f}) (5%-perc: {descr["5%"]:.
↳2f}, 95%-perc: {descr["95%"]:.2f})")

```

/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_2486/404798608.py:4:
PerformanceWarning: DataFrame is highly fragmented. This is usually the result
of calling `frame.insert` many times, which has poor performance. Consider
joining all columns at once using `pd.concat(axis=1)` instead. To get a de-
fragmented frame, use `newframe = frame.copy()`

```

vmax = np.percentile(df_p.assign(t_div_60=df_p[t] / 60)['t_div_60'].values,
95) * 3

```

```

median (time_total): 20.60 (25%-perc: 15.08, 75%-perc: 30.74)

```

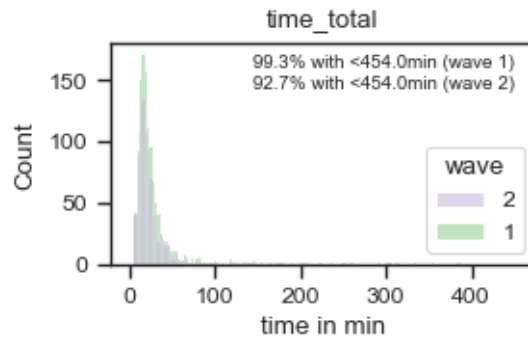
	time_trainingGame	time_training	time_spam	time_spam18dots \
count	2131.000000	2131.000000	2131.000000	1831.000000
mean	2.263890	101.814907	52.473025	1.982186
std	58.441322	1527.680292	1167.174120	32.867415
min	0.083333	0.300000	0.283333	-536.233333
25%	0.366667	1.433333	1.083333	0.816667
50%	0.533333	2.200000	1.600000	1.233333
75%	0.800000	3.516667	2.416667	1.883333
max	2631.750000	44530.183333	46892.716667	1298.333333

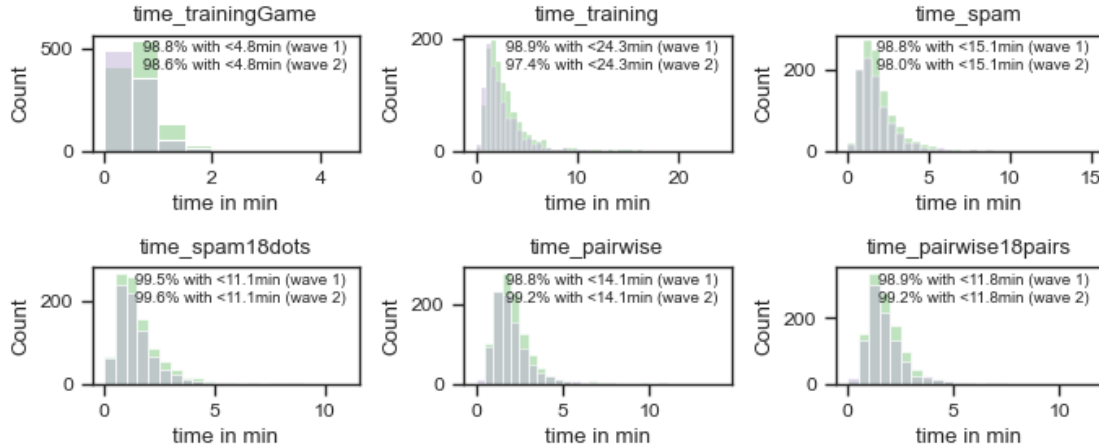
	time_pairwise	time_pairwise18pairs
count	2131.000000	2131.000000
mean	3.152636	2.682739
std	29.740875	29.302688
min	0.350000	0.283333

25%	1.366667	1.200000
50%	1.866667	1.600000
75%	2.533333	2.183333
max	1348.666667	1348.550000

	time_trainingGame	time_training	time_spam	time_spam18dots \
count	2131.000000	2.131000e+03	2.131000e+03	1831.000000
mean	135.833412	6.108894e+03	3.148382e+03	118.931185
std	3506.479304	9.166082e+04	7.003045e+04	1972.044882
min	5.000000	1.800000e+01	1.700000e+01	-32174.000000
25%	22.000000	8.600000e+01	6.500000e+01	49.000000
50%	32.000000	1.320000e+02	9.600000e+01	74.000000
75%	48.000000	2.110000e+02	1.450000e+02	113.000000
max	157905.000000	2.671811e+06	2.813563e+06	77900.000000

	time_pairwise	time_pairwise18pairs
count	2131.000000	2131.000000
mean	189.158142	160.964336
std	1784.452503	1758.161257
min	21.000000	17.000000
25%	82.000000	72.000000
50%	112.000000	96.000000
75%	152.000000	131.000000
max	80920.000000	80913.000000



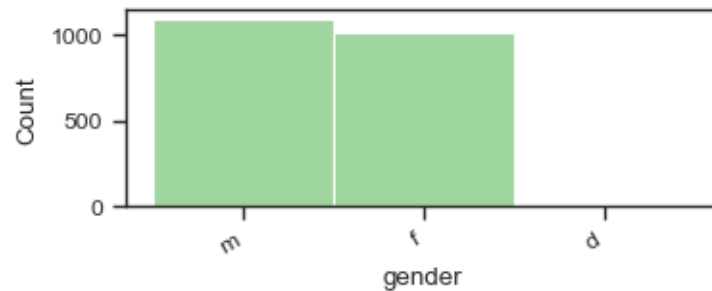


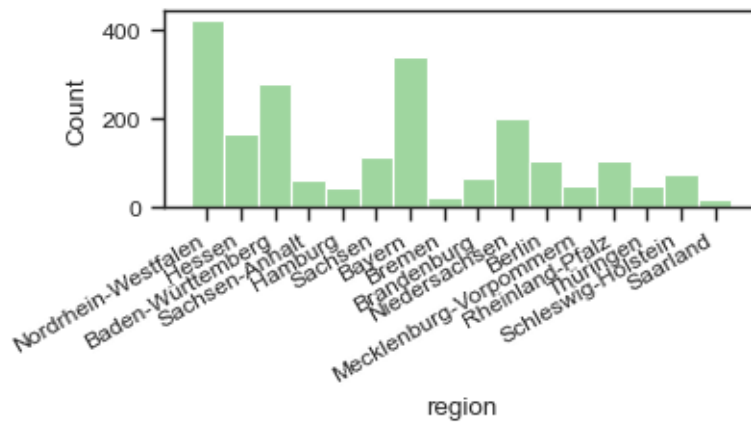
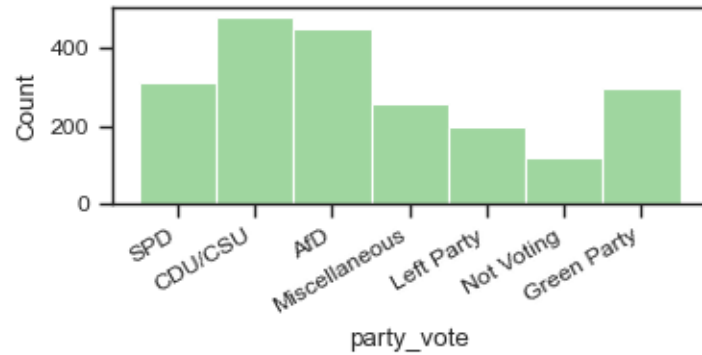
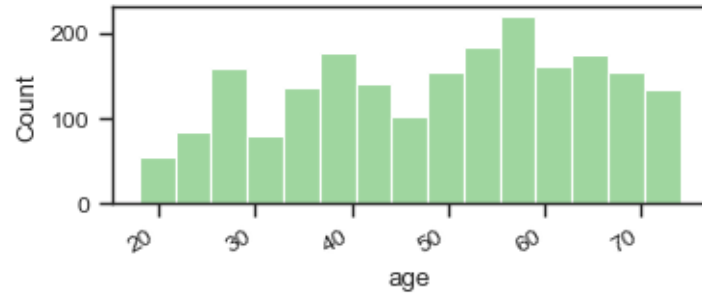
Note: some of the times are negative and should not be negative. This is probably because participants re-loaded the page and reloading overwrites the visited time stamp in the dataset.

2.1 Demographics

2.1.1 Age Gender Region (bilendi meta-data)

```
[7]: # %%
demo_cols = ["gender", "age", "party_vote", "region"] # participant-level
↳ constants
for d in demo_cols:
    fig = plt.figure(figsize=(4,1.5))
    ax = plt.axes()
    sns.histplot(df_p[d], ax=ax, color=cmapWave[1])
    fig.autofmt_xdate()
```





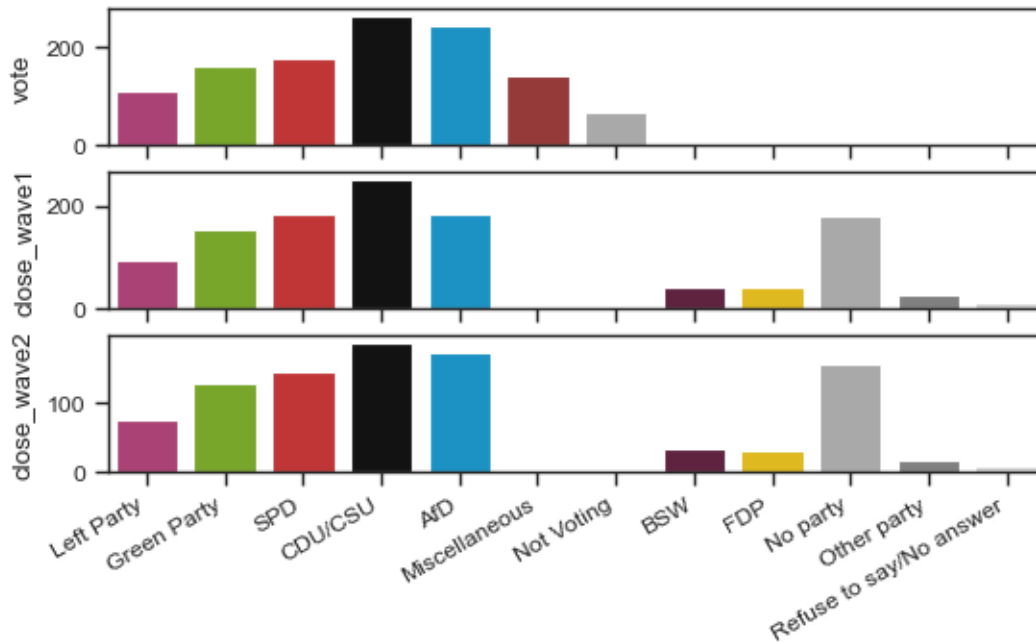
2.1.2 Parties / political identity

```
[8]: # %%
party_cmap["Miscellaneous"] = "brown"
party_cmap["Not Voting"] = "darkgrey"
fig, axs = plt.subplots(3,1, sharex=True)
```

```

for d, w, name, ax in zip(["party_vote", "party_close", "party_close"],
    ↳ [1,1,2], ["vote", "close_wave1", "close_wave2"], axs.flatten()):
    sns.barplot(df_p.loc[df_p.wave==w, d].value_counts()[parties_vote if "vote"
    ↳ in d else parties_full].reset_index(), ax=ax, y="count", x=d, hue=d,
    ↳ palette=party_cmap)
    fig.autofmt_xdate()
    ax.set_ylabel(name)
    ax.set_xlabel("")

```

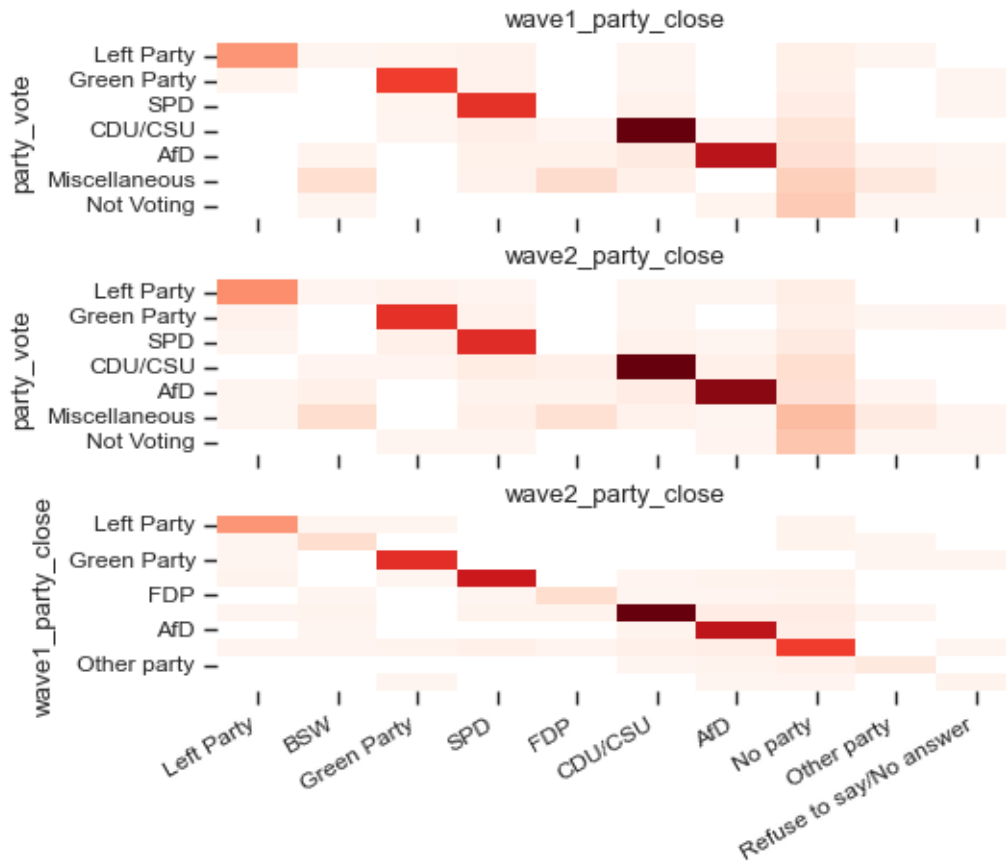


```

[9]: # %%
fig, axs = plt.subplots(3,1, figsize=(14/2.54, 12/2.54), sharex=True,
    ↳ sharey=False)
for (d1, d2), ax in zip(combinations(["party_vote", "wave1_party_close",
    ↳ "wave2_party_close"], 2), axs):

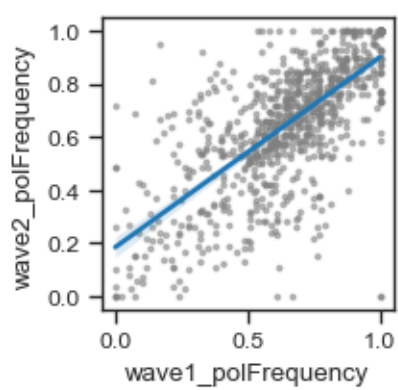
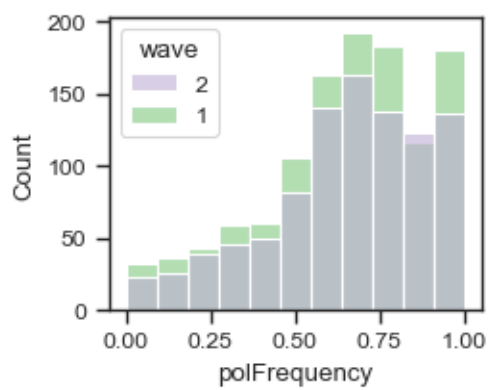
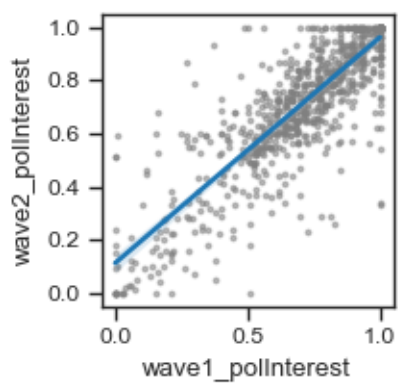
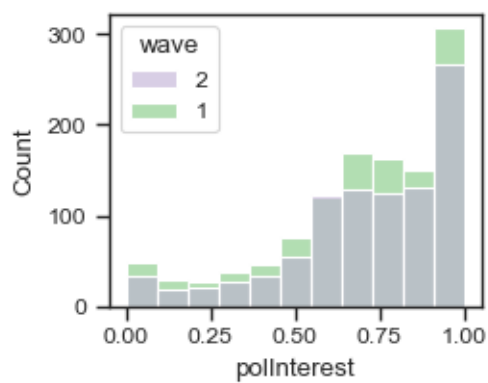
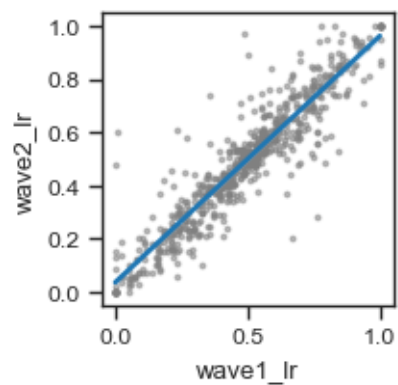
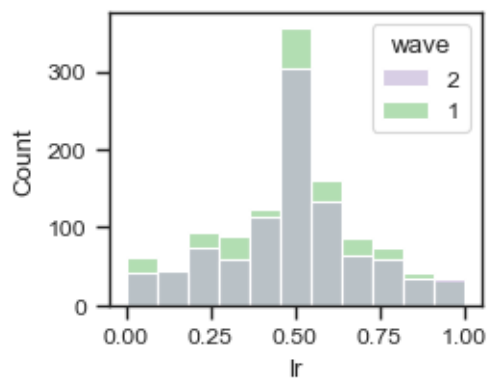
    sns.heatmap(df_p_bothwaves[[d1,d2,"age"]].pivot_table(index=d1,columns=d2,
    ↳ aggfunc="count",values="age").loc[parties_full if "wave" in d1 else
    ↳ parties_vote, parties_full if "wave" in d2 else parties_vote], cmap="Reds",
    ↳ ax=ax, cbar=False,)
    # ax.set_aspect("equal")
    ax.set_title(d2)
    # ax.set_ylabel("")
axs[-1].set_xlabel("")
fig.autofmt_xdate(rotation=30)
fig.tight_layout()

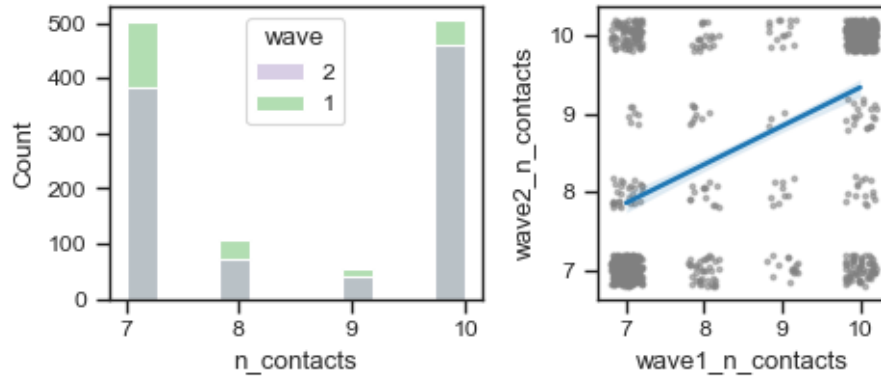
```

```
[12]: # %%
for var in ["lr", "polInterest", "polFrequency", "n_contacts"]:
    fig, axs = plt.subplots(1,2, figsize=(14/2.54,5/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6,
    ↪palette=cmapWave, hue_order=[2,1], bins=11)
    sns.regplot(df_p_bothwaves[["wave2_"+var, "wave1_"+var]], x="wave1_"+var,
    ↪y="wave2_"+var, ax=axs[1], scatter_kws={"s":3, "alpha":0.5, "color":"grey"},
    ↪y_jitter=0.2*(var=="n_contacts"), x_jitter=0.2*(var=="n_contacts"))
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
```

```
correlation wave 2 wave 1 lr: 0.9285788569852925
correlation wave 2 wave 1 polInterest: 0.8563886946865267
correlation wave 2 wave 1 polFrequency: 0.7105563299702032
correlation wave 2 wave 1 n_contacts: 0.48829717501188447
```





2.1.3 Polarisation

```
[13]: # %%
for var in ["P_tot"]+[f"P_{q}" for q in questions_sc]:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6,
    ↪palette=cmapWave, hue_order=[2,1], bins=11)
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],
    ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
descr1P = df_p[["P_tot"]+[f"P_{q}" for q in questions_sc]].describe()
display(descr1P)
```

```
correlation wave 2 wave 1 P_tot: 0.5197140603942746
correlation wave 2 wave 1 P_climate_concern: 0.406655671502216
correlation wave 2 wave 1 P_gay_marriage: 0.40457221775802465
correlation wave 2 wave 1 P_rights_indep_integration: 0.359740983017458
correlation wave 2 wave 1 P_econ_inequality: 0.35208645451441983
correlation wave 2 wave 1 P_regulate_internet: 0.32835752880320485
correlation wave 2 wave 1 P_east_germans: 0.3575416282293902
```

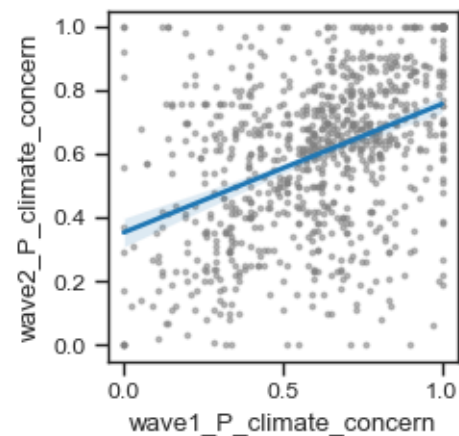
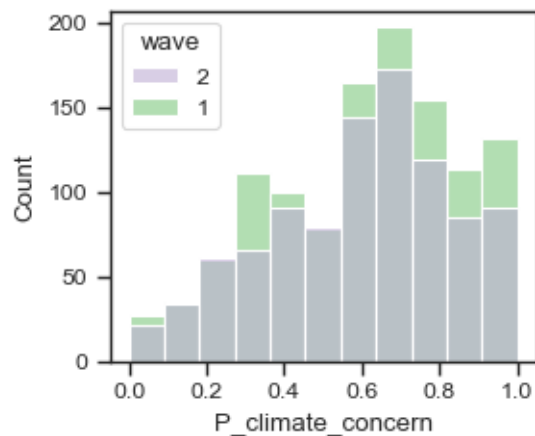
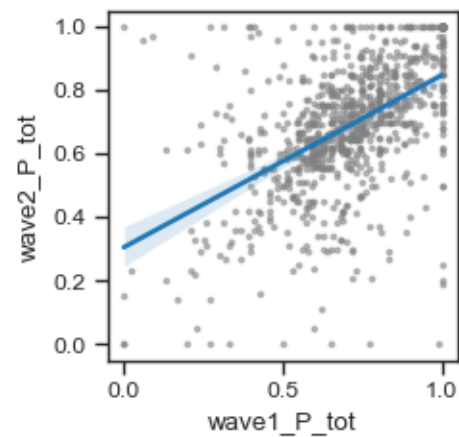
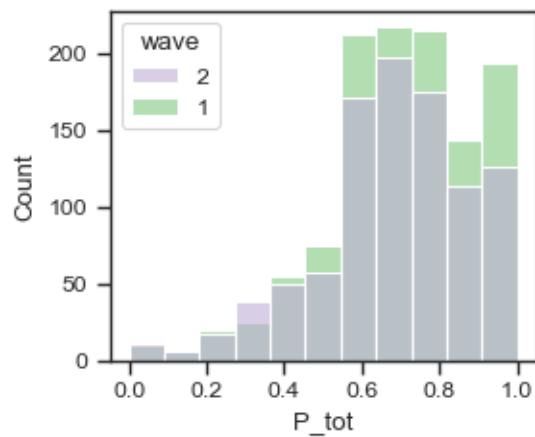
	P_tot	P_climate_concern	P_gay_marriage \
count	2131.000000	2131.000000	2131.000000
mean	0.694904	0.601694	0.538888
std	0.196952	0.242884	0.241068
min	0.000000	0.000000	0.000000
25%	0.590000	0.420000	0.370000
50%	0.700000	0.630000	0.570000
75%	0.830000	0.770000	0.700000
max	1.000000	1.000000	1.000000

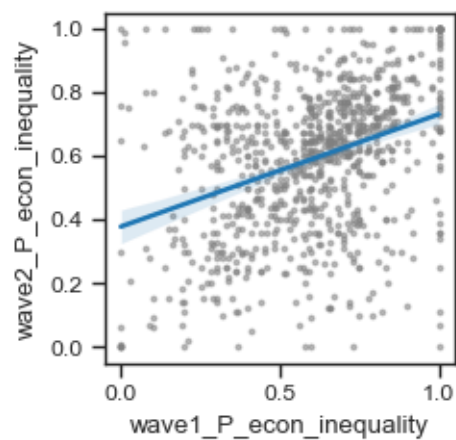
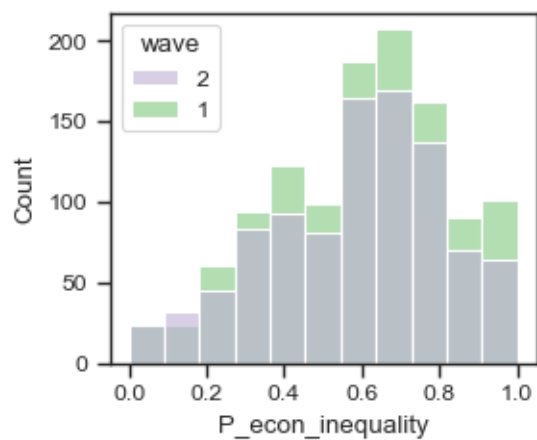
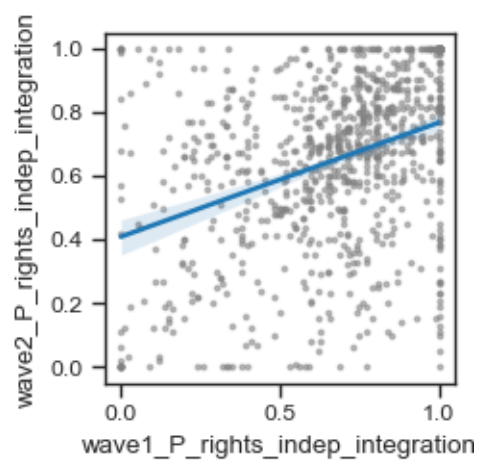
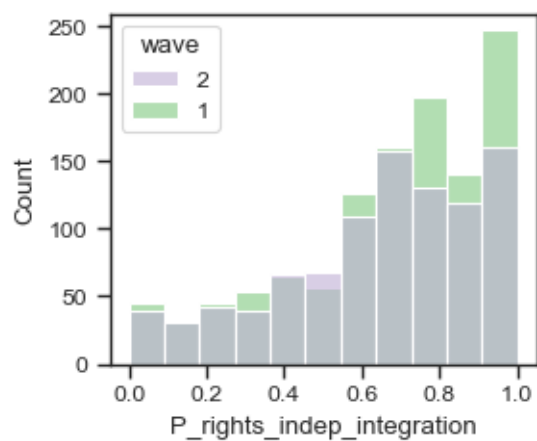
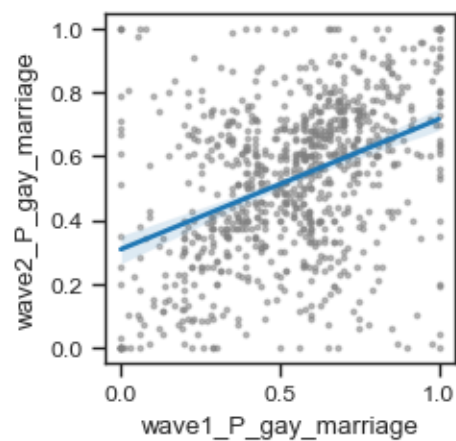
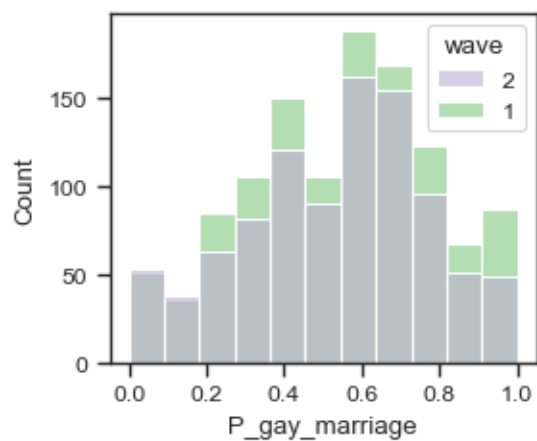
P_rights_indep_integration P_econ_inequality P_regulate_internet \

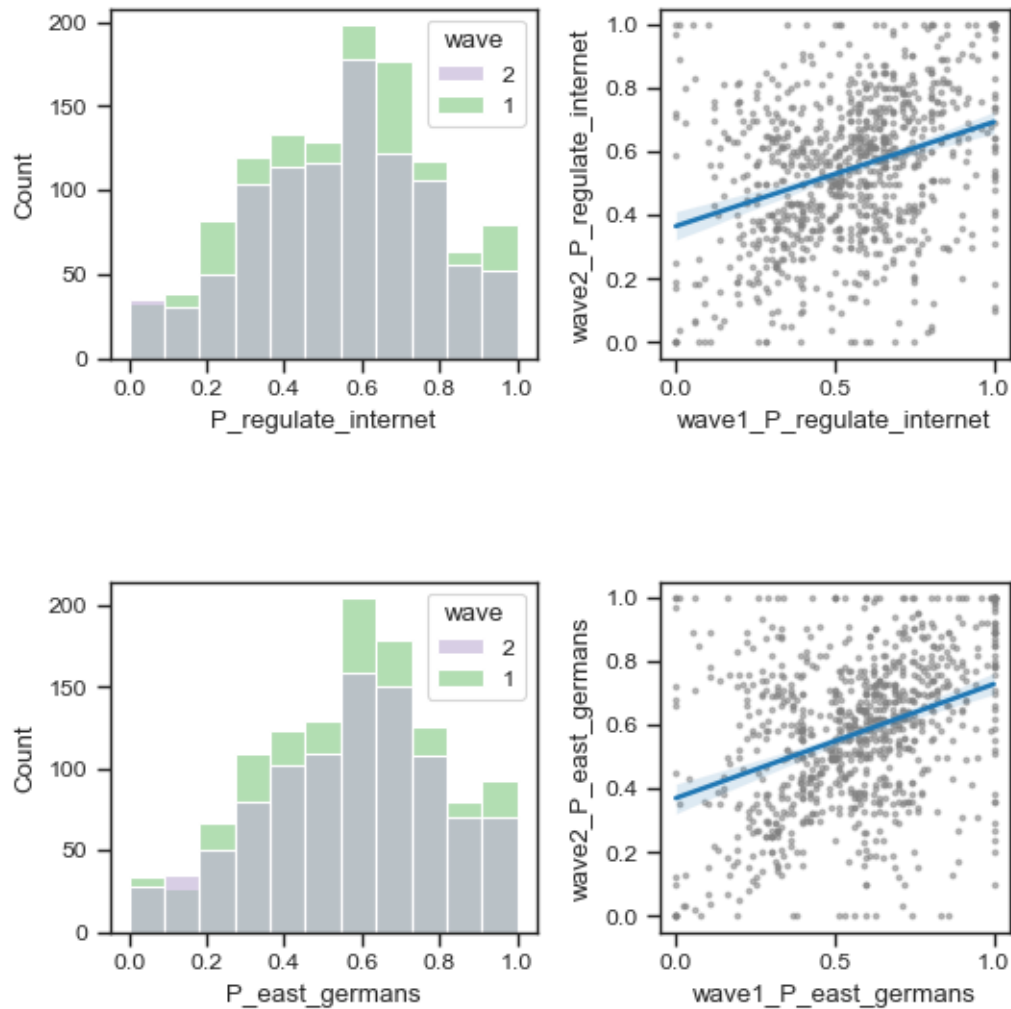
count	2131.000000	2131.000000	2131.000000
mean	0.660699	0.591764	0.546391
std	0.264068	0.227786	0.230033
min	0.000000	0.000000	0.000000
25%	0.510000	0.430000	0.380000
50%	0.710000	0.620000	0.570000
75%	0.860000	0.750000	0.700000
max	1.000000	1.000000	1.000000

P_east_germans

count	2131.000000
mean	0.570601
std	0.233402
min	0.000000
25%	0.400000
50%	0.590000
75%	0.730000
max	1.000000

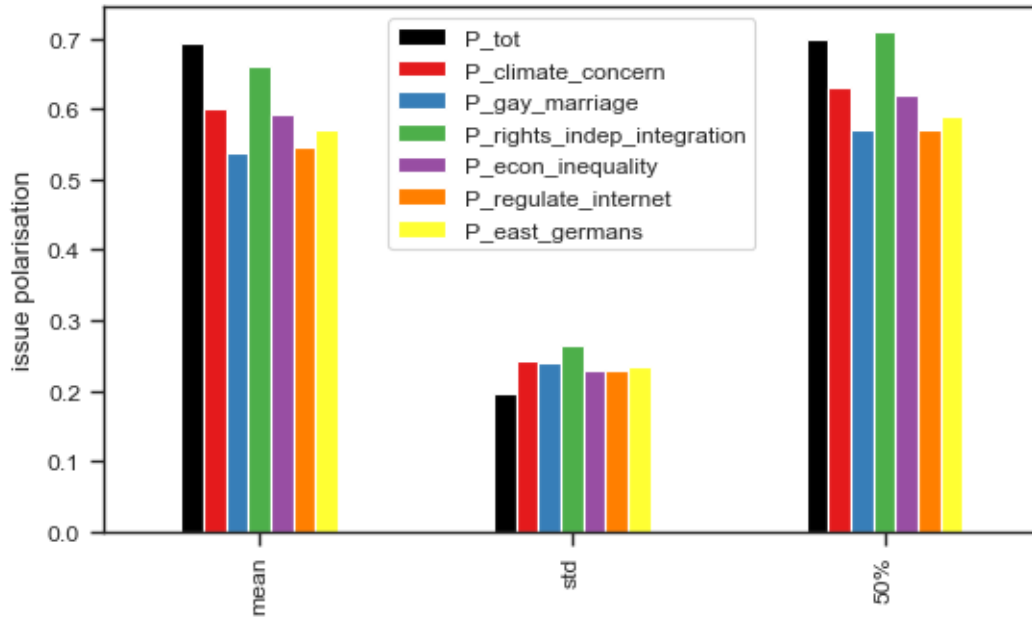






```
[16]: # %%
descriP.loc[["mean", "std", "50%"]].plot.bar(color=["k"]+list(cmapQuestions.
↪values()))
plt.ylabel("issue polarisation")
```

```
[16]: Text(0, 0.5, 'issue polarisation')
```



2.2 Issue Importance

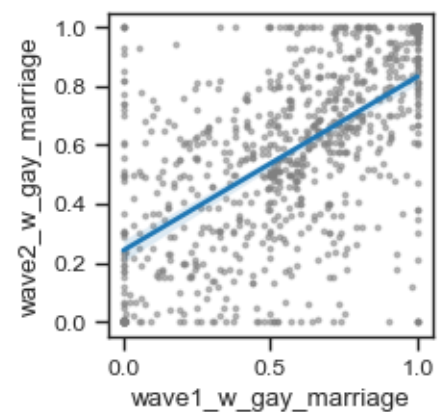
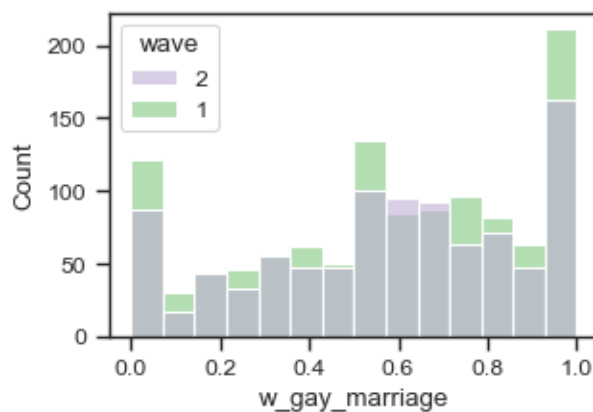
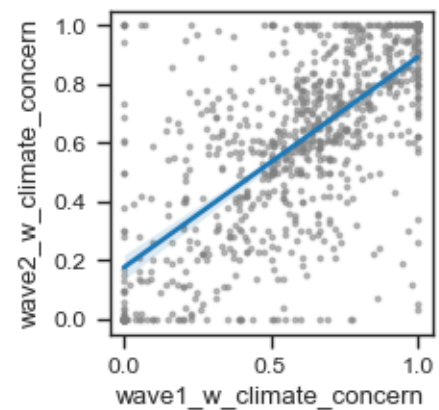
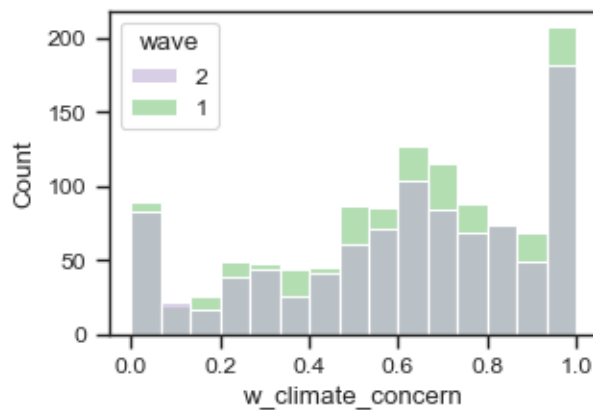
```
[19]: # %%
for var in [f"w_{q}" for q in questions_sc]:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6,
    ↪palette=cmapWave, hue_order=[2,1])
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],
    ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()
descr1w = df_p[[f"w_{q}" for q in questions_sc]].describe()
display(descr1w)
```

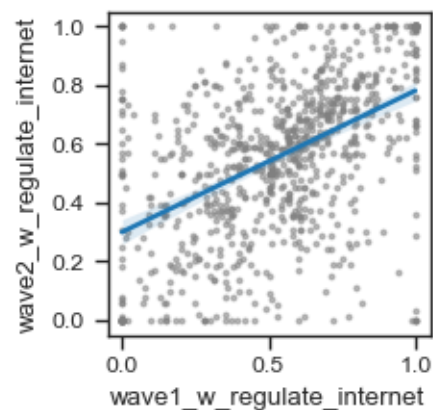
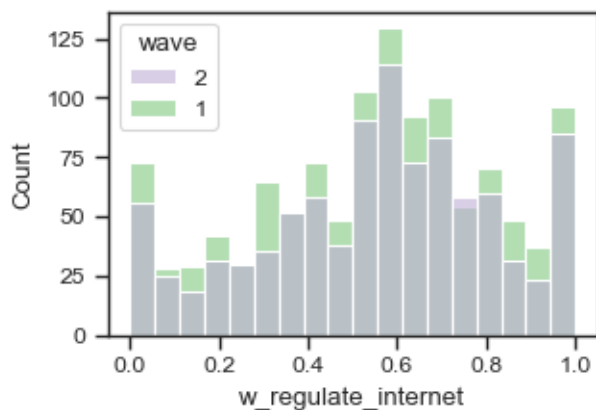
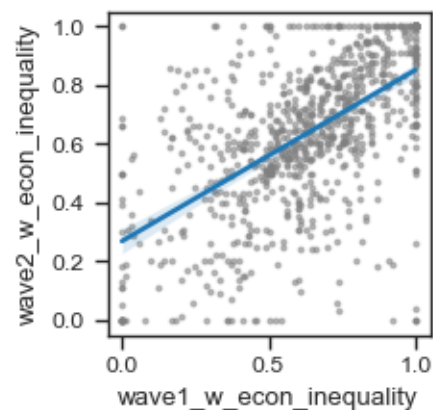
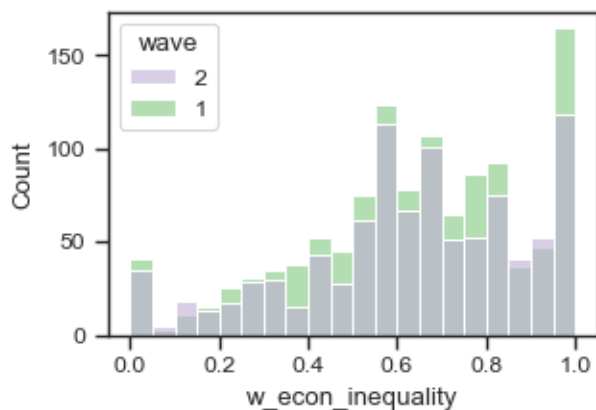
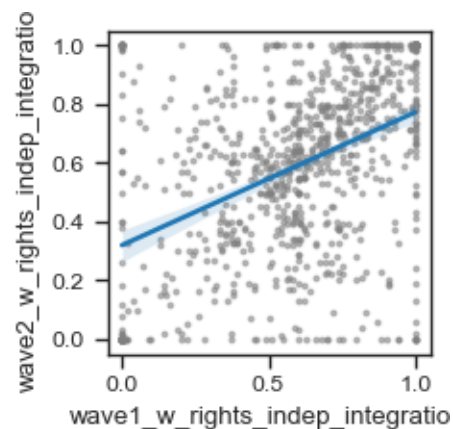
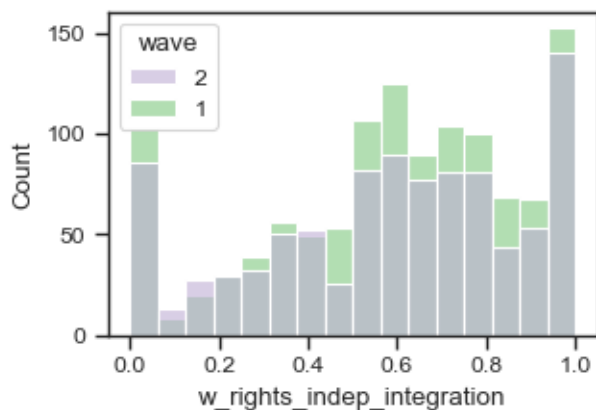
```
correlation wave 2 wave 1 w_climate_concern: 0.6900490845651501
correlation wave 2 wave 1 w_gay_marriage: 0.6066359275019365
correlation wave 2 wave 1 w_rights_indep_integration: 0.434059445041478
correlation wave 2 wave 1 w_econ_inequality: 0.5853821243834351
correlation wave 2 wave 1 w_regulate_internet: 0.48399610550840083
correlation wave 2 wave 1 w_east_germans: 0.6578730519621234
```

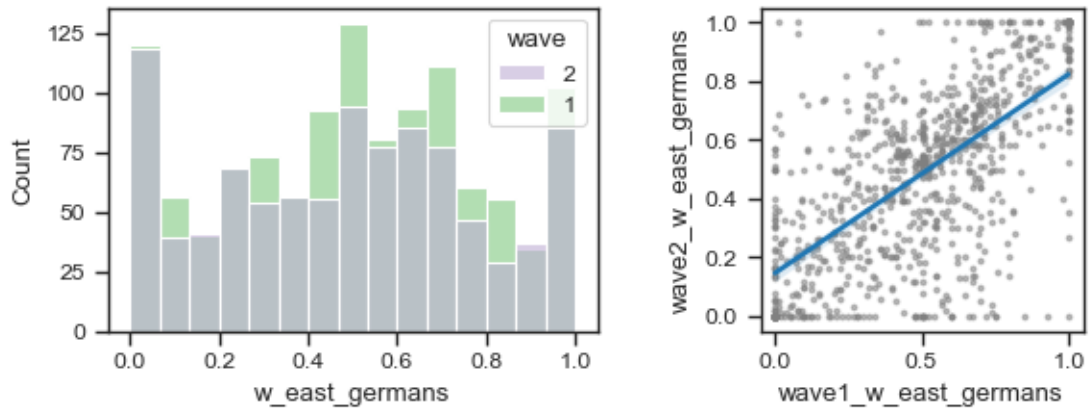
	w_climate_concern	w_gay_marriage	w_rights_indep_integration \
count	2131.000000	2131.000000	2131.000000
mean	0.610000	0.579620	0.590465
std	0.299637	0.308808	0.292577

min	0.000000	0.000000	0.000000
25%	0.410000	0.355000	0.390000
50%	0.650000	0.610000	0.620000
75%	0.845000	0.830000	0.810000
max	1.000000	1.000000	1.000000

	w_econ_inequality	w_regulate_internet	w_east_germans
count	2131.000000	2131.000000	2131.000000
mean	0.636283	0.549587	0.488381
std	0.255775	0.269327	0.293636
min	0.000000	0.000000	0.000000
25%	0.500000	0.370000	0.260000
50%	0.650000	0.580000	0.500000
75%	0.830000	0.740000	0.700000
max	1.000000	1.000000	1.000000

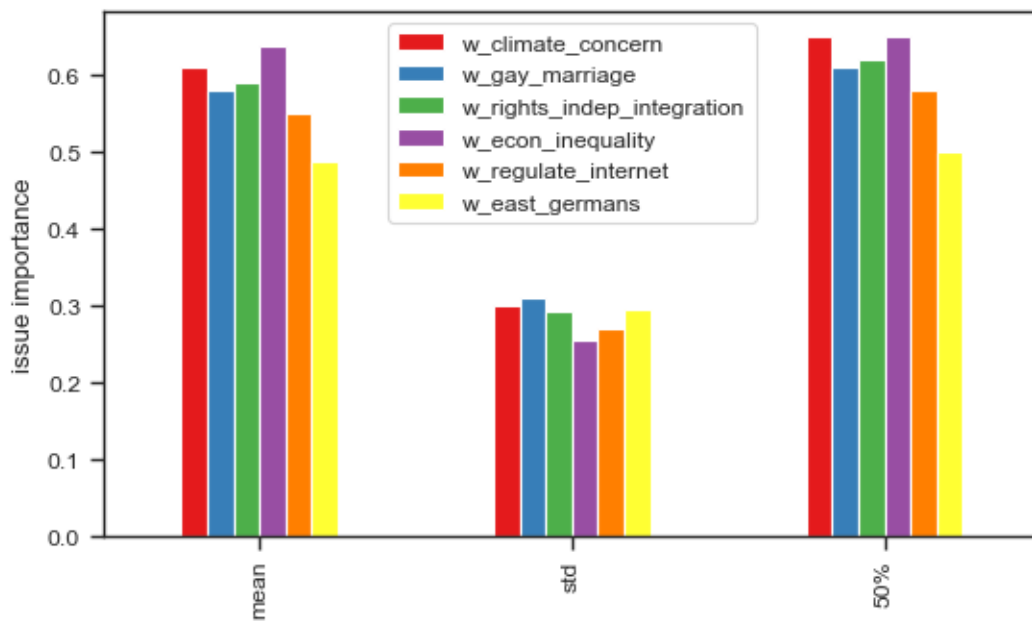






```
[20]: # %%
descriw.loc[["mean", "std", "50%"]].plot.bar(color=cmapQuestions.values())
plt.ylabel("issue importance")
```

```
[20]: Text(0, 0.5, 'issue importance')
```



```
[21]: # %%
df_p[[f"w_{q}" for q in questions_sc]].describe()
```

```
[21]:      w_climate_concern  w_gay_marriage  w_rights_indep_integration \
count      2131.000000      2131.000000      2131.000000
```

mean	0.610000	0.579620	0.590465
std	0.299637	0.308808	0.292577
min	0.000000	0.000000	0.000000
25%	0.410000	0.355000	0.390000
50%	0.650000	0.610000	0.620000
75%	0.845000	0.830000	0.810000
max	1.000000	1.000000	1.000000

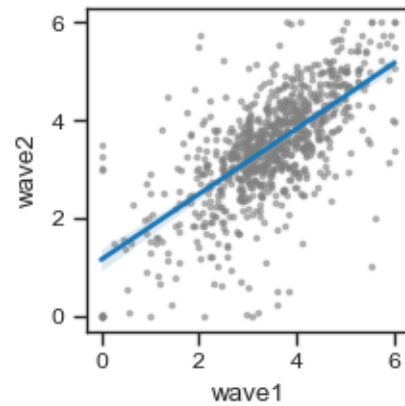
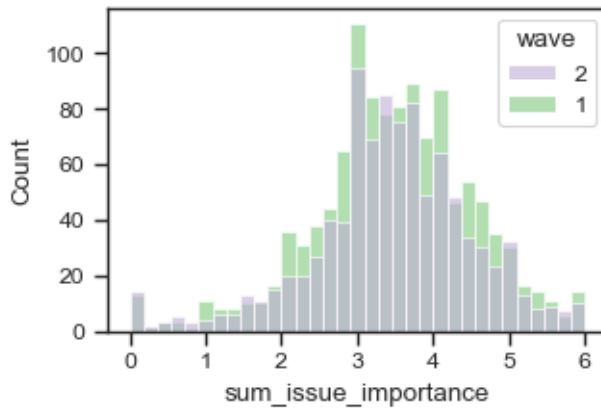
	w_econ_inequality	w_regulate_internet	w_east_germans
count	2131.000000	2131.000000	2131.000000
mean	0.636283	0.549587	0.488381
std	0.255775	0.269327	0.293636
min	0.000000	0.000000	0.000000
25%	0.500000	0.370000	0.260000
50%	0.650000	0.580000	0.500000
75%	0.830000	0.740000	0.700000
max	1.000000	1.000000	1.000000

```
[22]: # %%
var = "sum_issue_importance"
fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
df_p[var] = df_p[[f"w_{q}" for q in questions_sc]].sum(axis=1)
sns.histplot(df_p,x=var, hue="wave", ax=axs[0], alpha=0.6, palette=cmapWave,
             hue_order=[2,1])
aa = df_p.pivot_table(index="id", columns="wave", values=var).rename(columns={1:
             "wave1", 2:"wave2"})
sns.regplot(aa, x="wave1", y="wave2", ax=axs[1], scatter_kws={"s":3, "alpha":0.
             "color":"grey"})
axs[1].set_aspect("equal")
print(f"correlation wave 2 wave 1 {var}: {aa.corr().values[1,0]}")
fig.tight_layout()
display(df_p[var].describe())
```

correlation wave 2 wave 1 sum_issue_importance: 0.6638441117194276

count	2131.000000
mean	3.454336
std	1.064184
min	0.000000
25%	2.920000
50%	3.470000
75%	4.120000
max	6.000000

Name: sum_issue_importance, dtype: float64



2.3 Voter Sympathy/Likeability

```
[23]: # %%

N = len(partiesVars)
ncols = 2
nrows = (N + 1) // ncols # ceiling division

fig, axs = plt.subplots(
    nrows, ncols,
    figsize=(16/2.54, 12/2.54),
    sharex=True, sharey=True
)
axs_flat = axs.flatten()

for i, p in enumerate(partiesVars):
    ax = axs_flat[i]
    sns.histplot(
        data=df_p,
        x=f"sym_{p}",
        hue="party_close",
        ax=ax,
        alpha=0.6,
        palette=party_cmap,
        multiple="stack",
        bins=np.linspace(-0.01, 1.01, 21),
        legend=(i == N - 1),
        linewidth=0,
        hue_order=parties_full,
    )
    ax.set_title(f"towards {p} voter", x=0.95, y=0.8,
```

```

        color=party_cmap[p], va="top", ha="right")
    ax.set_ylabel("")
    ax.set_xlabel("sympathy towards typical voter" if i >= N - ncols else "")

# Hide unused axes (last cell if N is odd)
    for ax in axs_flat[N:]:
        ax.axis("off")

    ax = axs_flat[N-1]
    sns_legend = ax.get_legend()
    handles = sns_legend.legend_handles
    labels = [t.get_text() for t in sns_legend.get_texts()]
    sns_legend.remove()

# Hide the last axes but keep it as a legend host
    empty_ax = axs_flat[N]
    empty_ax.axis("off")

    empty_ax.legend(
        handles, labels,
        bbox_to_anchor=(0.05, 0.5),
        ncol=2,
        fontsize=6,
        title="party id",
        title_fontsize=6,
        loc="center left",
        borderaxespad=0,
    )

    display(df_p[[f"sym_{p}" for p in partiesVars]].describe())

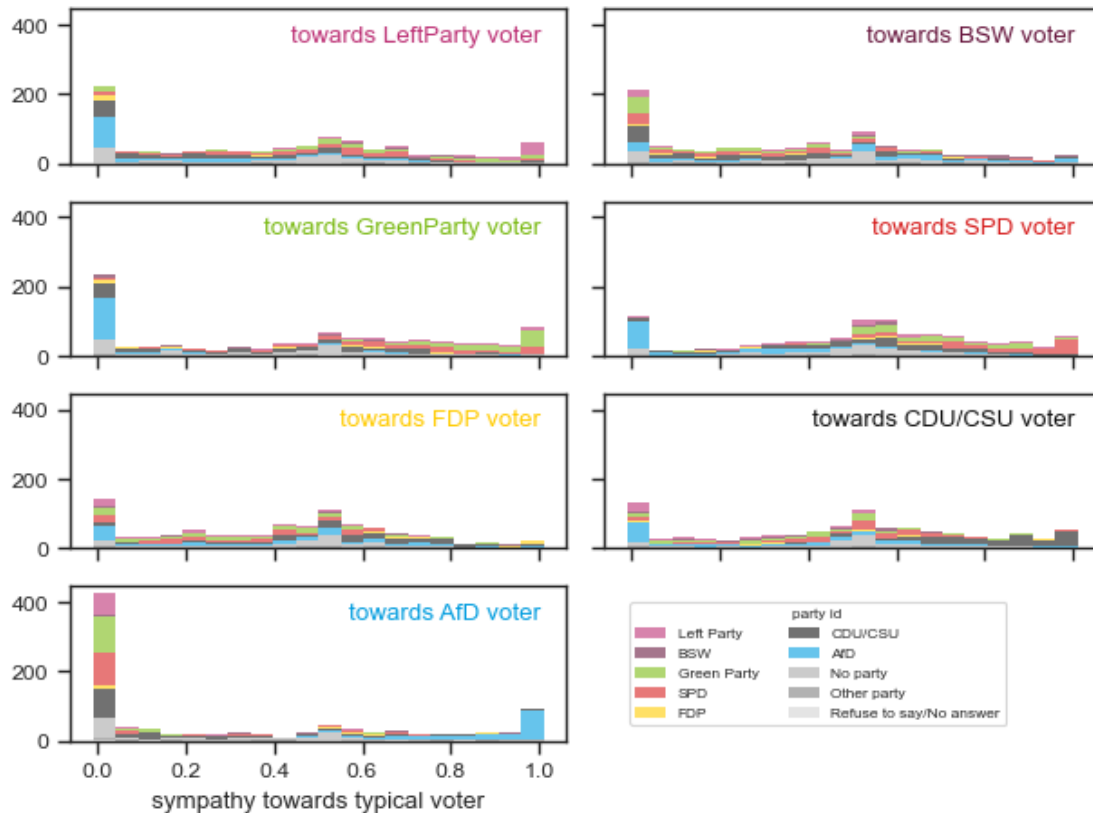
    fig.tight_layout()
    plt.savefig("sympathy.png", dpi=600)

```

	sym_LeftParty	sym_BSW	sym_GreenParty	sym_SPD	sym_FDP \
count	961.000000	961.000000	961.000000	961.000000	961.000000
mean	0.400708	0.362487	0.449740	0.518803	0.409501
std	0.316579	0.288055	0.345557	0.283031	0.269797
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.070000	0.080000	0.050000	0.350000	0.190000
50%	0.420000	0.360000	0.500000	0.560000	0.440000
75%	0.630000	0.570000	0.740000	0.710000	0.600000
max	1.000000	1.000000	1.000000	1.000000	1.000000

	sym_CDU/CSU	sym_AfD
count	961.000000	961.000000
mean	0.478127	0.320437
std	0.296390	0.365198

min	0.000000	0.000000
25%	0.250000	0.000000
50%	0.510000	0.120000
75%	0.690000	0.630000
max	1.000000	1.000000



3 Training

```
[24]: var = "attemptsPractice"
print(f"Fraction of training fails in wave 1: {sum((df_p.
↪ wave==1)*(df_p[var]==-999))/sum(df_p.wave==1):.4f}")
# df_p[["wave", var]].value_counts().reset_index().sort_values("wave")
```

Fraction of training fails in wave 1: 0.2043

```
[26]: # %%
for var in ["attemptsPractice"]:
    fig, ax = plt.subplots(1,1, figsize=(3,2))
```

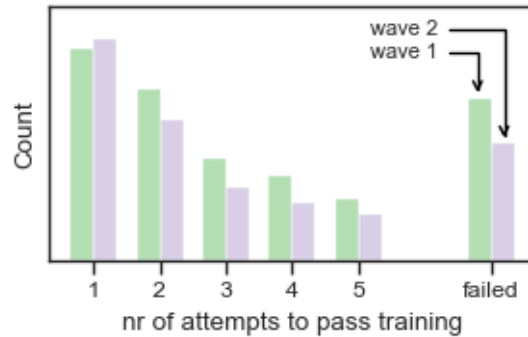
```

sns.histplot(df_p.replace({-999:7}), x=var, hue="wave", ax=ax,
↪palette=cmapWave, bins=np.array([[k-0.35, k+0.35] for k in range(1,8)]).
↪flatten(), alpha=0.6, multiple="dodge", legend=False, stat="count")
ax.set_xticks([1,2,3,4,5, 7])
ax.set_yticks([])
ax.set_xticklabels(["1","2","3","4","5", "failed"])
ax.set_xlabel("nr of attempts to pass training")
# wave 1 -> left sub-bar of the "failed" group
ymax = ax.get_ylim()[1]
# wave 1 -> left sub-bar (shorter elbow, drops in closer to the label)
ax.annotate(
    "wave 1",
    xy=(6.8, df_p.loc[df_p.wave==1, var].value_counts()[-999]),
↪xycoords="data",          # arrow tip on left bar
    xytext=(6.2, ymax * 0.9), textcoords="data",
    fontsize=8, ha="right", va="center",
    arrowprops=dict(
        arrowstyle="->", color="black", lw=1,
        connectionstyle="angle,angleA=0,angleB=90,rad=0"
    )
)

# wave 2 -> right sub-bar (longer horizontal run, sits above wave 1's label)
ax.annotate(
    "wave 2",
    xy=(7.2, df_p.loc[df_p.wave==2, var].value_counts()[-999]),
↪xycoords="data",          # arrow tip on right bar
    xytext=(6.2, ymax * 1), textcoords="data",
    fontsize=8, ha="right", va="center",
    arrowprops=dict(
        arrowstyle="->", color="black", lw=1,
        connectionstyle="angle,angleA=0,angleB=90,rad=0"
    )
)
ax.set_ylim(top=ymax * 1.1) # give headroom for the labels
print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
↪'wave1_'+var]].replace({-999:np.nan}).corr().values[1,0]}")
plt.tight_layout()
plt.savefig("figs/trainingattempts.png", dpi=600 )

```

correlation wave 2 wave 1 attemptsPractice: 0.20324312818343532

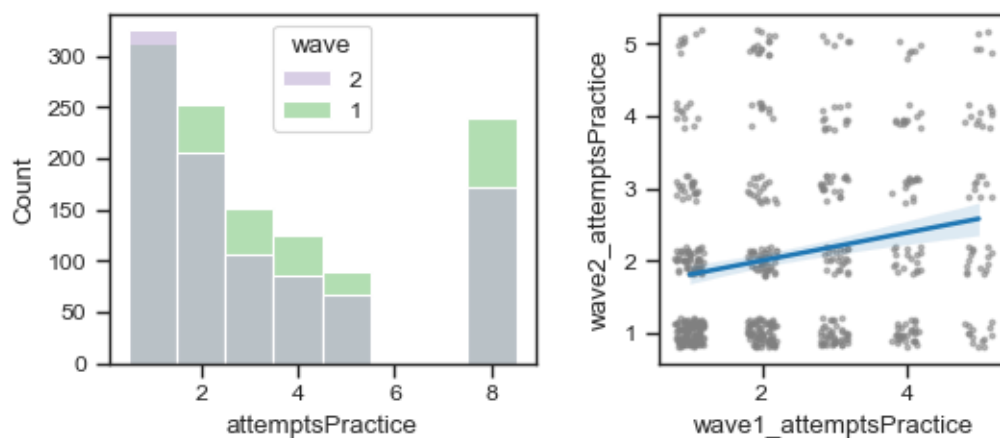


```
[27]: # %%
for var in ["attemptsPractice"]:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))

    sns.histplot(df_p.replace({-999:8}), x=var, hue="wave", ax=axs[0], bins=0.
    ↪5+np.arange(0,9), alpha=0.6, palette=cmapWave, hue_order=[2,1])
    sns.regplot(df_p_bothwaves[["wave1_"+var, "wave2_"+var]].replace({-999:np.
    ↪nan}), x="wave1_"+var, y="wave2_"+var, ax=axs[1], scatter_kws={"s":3,
    ↪"alpha":0.5, "color":"grey"}, y_jitter=0.2*(var=="attemptsPractice"),
    ↪x_jitter=0.2*(var=="attemptsPractice"))
    axs[1].set_aspect("equal")

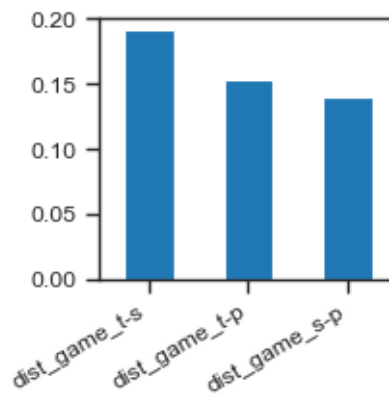
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].replace({-999:np.nan}).corr().values[1,0]}")
```

correlation wave 2 wave 1 attemptsPractice: 0.20324312818343532




```
[28]: # %%
display(df_p[[f"dist_game_{a[0]}-{b[0]}" for a,b in
↳ combinations(practice_game_dots, 2)]].describe())
fig = plt.figure(figsize=(2,2))
df_p[[f"dist_game_{a[0]}-{b[0]}" for a,b in combinations(practice_game_dots,
↳ 2)]].mean().plot.bar()
fig.autofmt_xdate()
```

	dist_game_t-s	dist_game_t-p	dist_game_s-p
count	2131.000000	2131.000000	2131.000000
mean	0.191509	0.152629	0.139701
std	0.091240	0.085501	0.080237
min	0.001818	0.001286	0.001286
25%	0.124870	0.089510	0.077762
50%	0.180742	0.135390	0.124734
75%	0.252008	0.198827	0.184760
max	0.501818	0.501818	0.501435

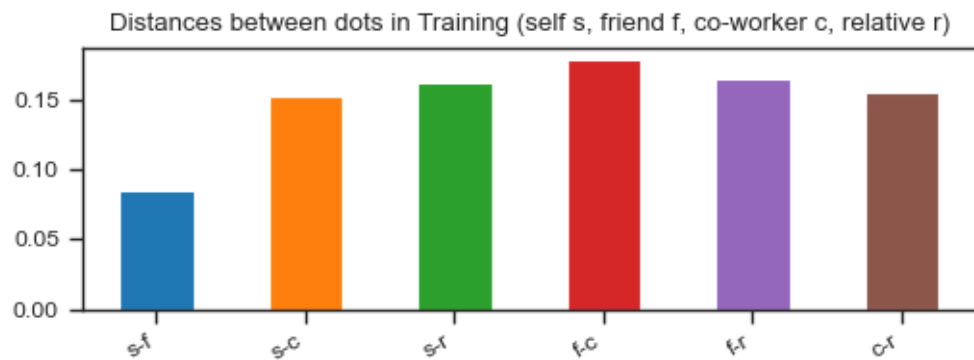


```
[29]: # %%
vars = [f"dist_practice_{a[0]}-{b[0]}" for a,b in
↳ combinations(practice_training_dots, 2)]
display(df_p[vars].describe())
fig = plt.figure(figsize=(6,2))
df_p[vars].mean().plot.bar(color=plt.get_cmap("tab10").colors)
plt.xticks(plt.gca().get_xticks(), labels=[s[-3:] for s in vars])
plt.title("Distances between dots in Training (self s, friend f, co-worker c,
↳ relative r)")
fig.autofmt_xdate()
```

	dist_practice_s-f	dist_practice_s-c	dist_practice_s-r \
count	2131.000000	2131.000000	2131.000000
mean	0.084429	0.151904	0.161647
std	0.054933	0.075922	0.082202

min	0.002875	0.014374	0.011499
25%	0.047304	0.093123	0.095583
50%	0.068188	0.137276	0.143576
75%	0.104395	0.194868	0.217880
max	0.425764	0.487494	0.501818

	dist_practice_f-c	dist_practice_f-r	dist_practice_c-r
count	2131.000000	2131.000000	2131.000000
mean	0.178163	0.164070	0.155472
std	0.078194	0.071335	0.074121
min	0.016109	0.010602	0.018408
25%	0.117632	0.110763	0.096466
50%	0.167312	0.154674	0.145028
75%	0.230376	0.207524	0.200532
max	0.501818	0.488367	0.489965



```
[30]: # %%
df_p[["wave", "passed_practice_sanity"]].replace({np.nan:"failed training"}).
↳groupby("wave")["passed_practice_sanity"].value_counts().reset_index()
```

```
[30]: wave passed_practice_sanity count
0      1                1.0      925
1      1      failed training      239
2      1                0.0        6
3      2                1.0      776
4      2      failed training      172
5      2                0.0       13
```

4 Mapping

```
[38]: wave = 2
ids = df_p.loc[df_p.wave==wave]["id"].sample(9).values

def plot_map(ax, x, q, pos_processed, wave):
    colors = colorsOrig if q==" else [x["player."+f"{'own2' if p=='self' else p}__{q}"].values[wave-1] for p in peeps]
    if q=="":
        ax.scatter([pos_processed[0,0]], [pos_processed[0,1]], c=colors[0], s=20, marker="X")
        ax.scatter(pos_processed[1:,0], pos_processed[1:,1], c=colors[1:], s=20, )
    else:
        ax.scatter([pos_processed[0,0]], [pos_processed[0,1]], c=colors[0], s=20, cmap=plt.get_cmap("coolwarm"), vmin=-100, vmax=100, marker="X")
        ax.scatter(pos_processed[1:,0], pos_processed[1:,1], c=colors[1:], s=20, cmap=plt.get_cmap("coolwarm"), vmin=-100, vmax=100, )
        ax.set_aspect("equal")
        ax.set_xlim(0, MAX_PIXELPOS)
        ax.set_ylim(0, MAX_PIXELPOS)
        ax.set_xticks([])
        ax.set_yticks([])

fig, axs = plt.subplots(3,3, figsize=(12/2.54, 12/2.54), sharex=True, sharey=True)
for id, ax in zip(ids, axs.flatten()):
    partic_data = df_orig.loc[(df_orig.wave==wave) & (df_orig.bilendi_id==id)]
    pos = json.loads(partic_data["player.positions"].values[0])
    pos_processed = {p["varname"].replace(" ", ""): np.array([p["x"], p["y"]]) for p in pos}
    pos_processed = np.array([[np.nan, np.nan] if k.replace(" ", "") not in pos_processed else pos_processed[k] for k in peeps])
    # pos_processed_dict = {k: [np.nan, np.nan] if k not in pos_processed else pos_processed[k] for k in peeps}

    plot_map(ax, partic_data, "", pos_processed, np.nan)
    ax.text(0.0, 1.02, f"id: {id} (w{wave})", transform=ax.transAxes, fontsize=5)
fig.tight_layout(h_pad=1, w_pad=1)
plt.show()
```

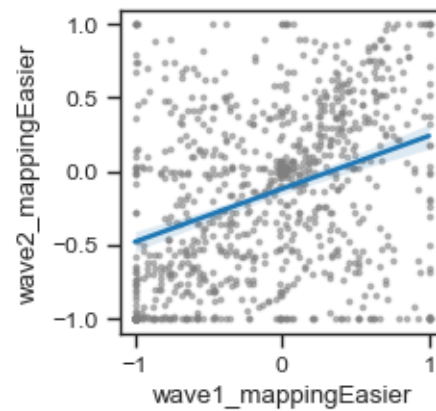
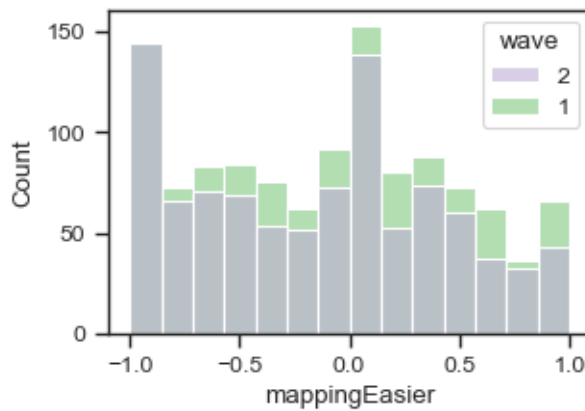
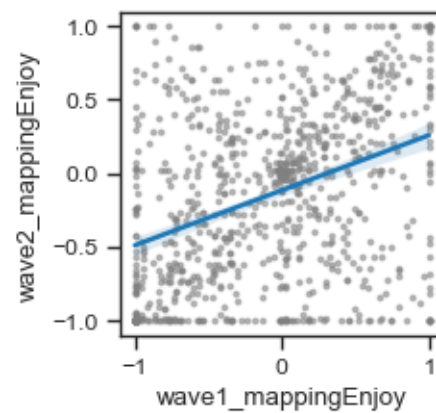
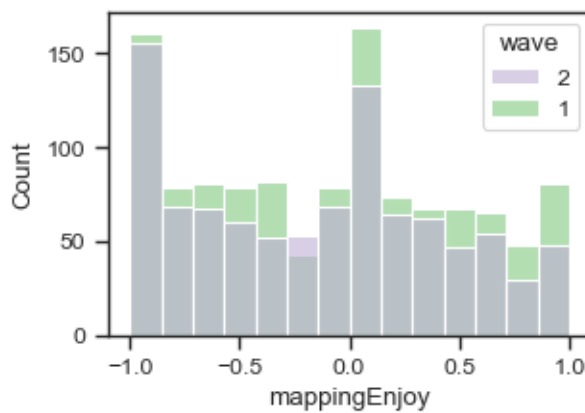


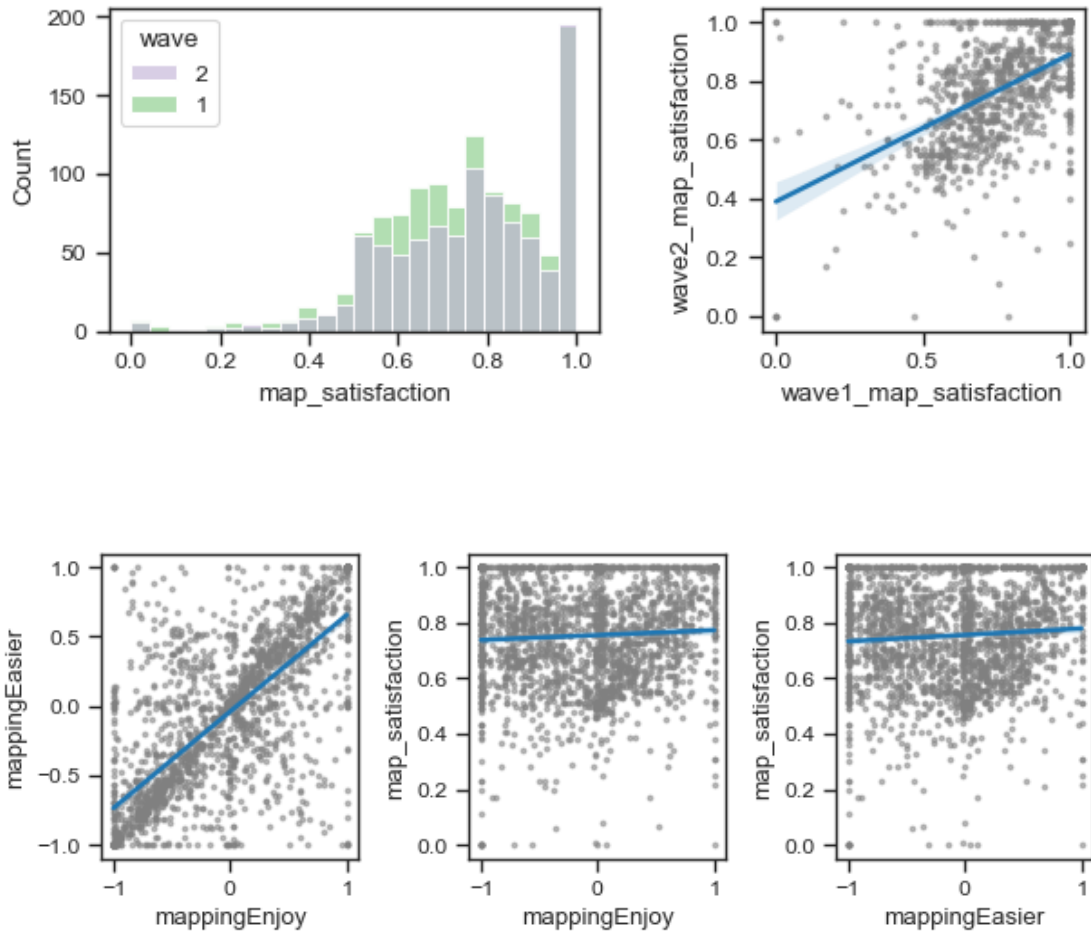
```
[40]: # %%
vars = ["mappingEnjoy", "mappingEasier", "map_satisfaction"]
for var in vars:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p,x=var, hue="wave", ax=axs[0], alpha=0.6,
    ↪palette=cmapWave, hue_order=[2,1])
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],
    ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

display(df_p[vars].describe())
fig, axs = plt.subplots(1,3, figsize=(16/2.54,6/2.54))
for (var1, var2), ax in zip(combinations(vars,2), axs.flatten()):
    sns.regplot(df_p, x=var1, y=var2, ax=ax, scatter_kws={"s":3, "alpha":0.5,
    ↪"color":"grey"})
fig.tight_layout()
```

correlation wave 2 wave 1 mappingEnjoy: 0.3746722332641922
correlation wave 2 wave 1 mappingEasier: 0.3559971135908032
correlation wave 2 wave 1 map_satisfaction: 0.5124366548145483

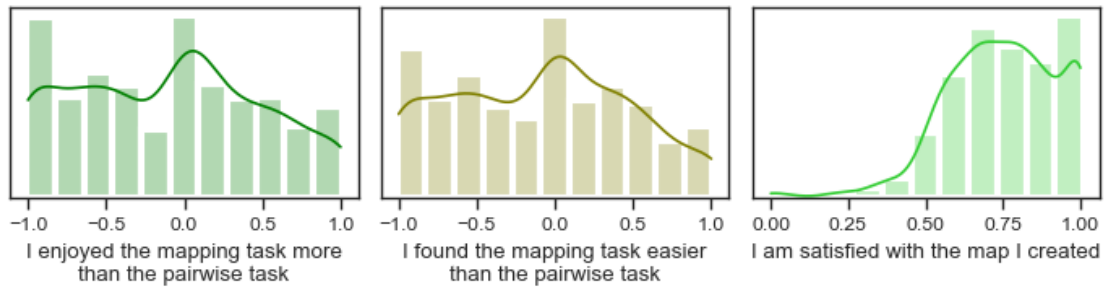
	mappingEnjoy	mappingEasier	map_satisfaction
count	2131.000000	2131.000000	2131.000000
mean	-0.119962	-0.121192	0.753867
std	0.593370	0.573330	0.185781
min	-1.000000	-1.000000	0.000000
25%	-0.640000	-0.620000	0.630000
50%	-0.040000	-0.040000	0.770000
75%	0.320000	0.320000	0.900000
max	1.000000	1.000000	1.000000





```
[41]: fig, axes = plt.subplots(1,3, figsize=(7, 2.), sharey=False, sharex=False)
vars = ["mappingEnjoy", "mappingEasier", "map_satisfaction"]
colors = dict(zip(vars, ["green", "olive", "limegreen"]))

for ax, col in zip(axes, vars):
    sns.histplot(df_p.query(f"wave==1")[col], color=colors[col], bins=11,
    ↪binrange=[-0.01,1.01] if "satis" in col else [-1.01,1.01], ax=ax, kde=True,
    ↪linewidth=3, alpha=0.3)
    ax.set_yticks([])
    ax.set_ylabel("responses" if "sim" in col else "")
    ax.set_xlabel(r"I enjoyed the mapping task more"+"\\n"than the pairwise task"
    ↪if "Enjoy" in col else (r"I found the mapping task easier"+"\\n"than the
    ↪pairwise task" if "Easier" in col else r"I am satisfied with the map I
    ↪created"+"\\n"))
plt.tight_layout()
plt.savefig("figs/mappingTaskEvaluation.png", dpi=600)
```

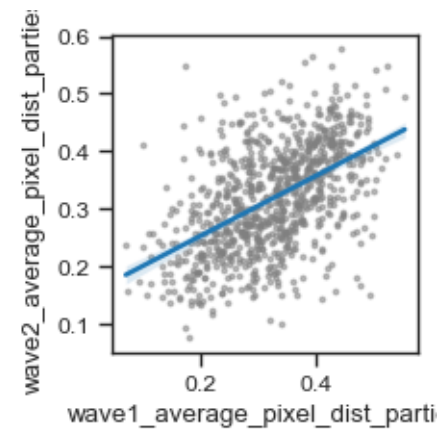
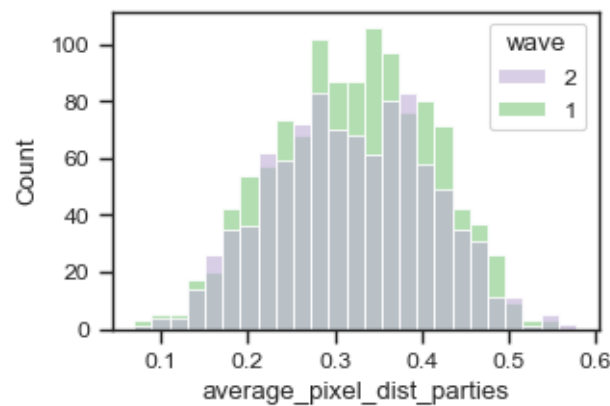
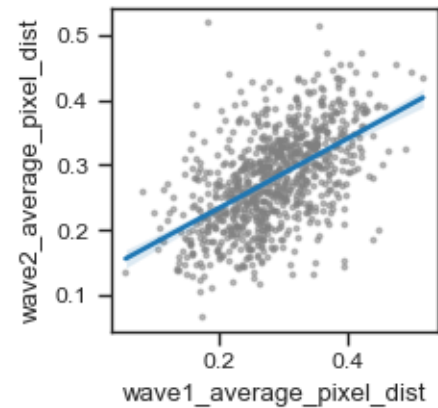
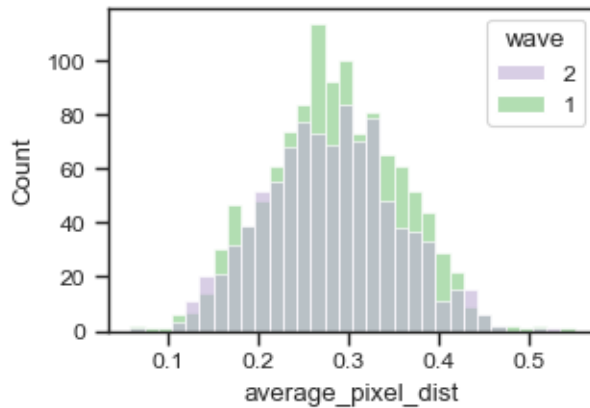


```
[42]: # %%
vars = ["average_pixel_dist", "average_pixel_dist_parties"]
for var in vars:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6,
    ↪palette=cmapWave, hue_order=[2,1])
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],
    ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

display(df_p[vars].describe())
```

```
correlation wave 2 wave 1 average_pixel_dist: 0.544919637973932
correlation wave 2 wave 1 average_pixel_dist_parties: 0.5180154828198414
```

	average_pixel_dist	average_pixel_dist_parties
count	2131.000000	2131.000000
mean	0.281627	0.320045
std	0.073028	0.089704
min	0.057501	0.068942
25%	0.230760	0.254271
50%	0.280162	0.320929
75%	0.331161	0.387069
max	0.549351	0.577399



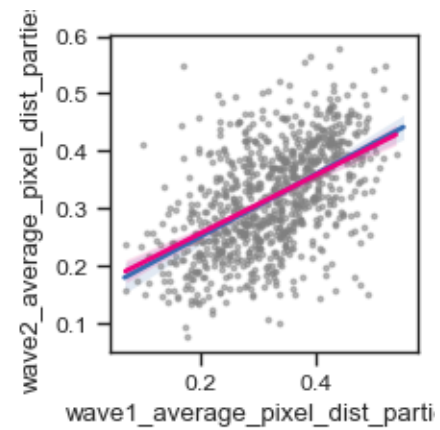
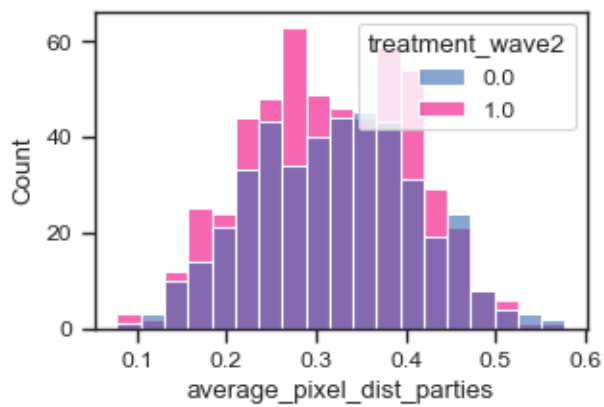
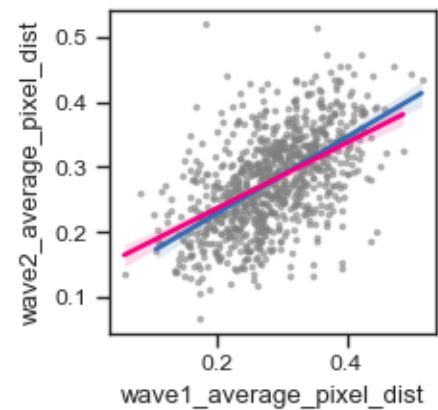
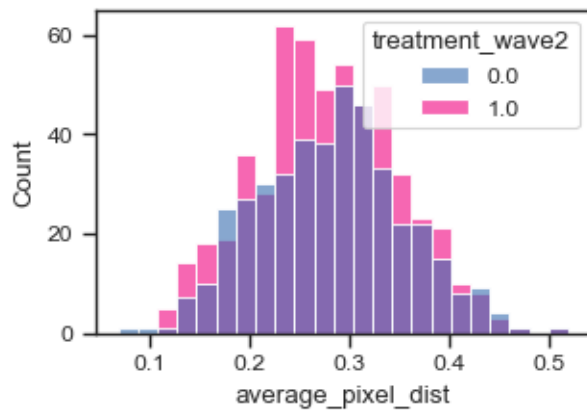
```
[50]: # %%
vars = ["average_pixel_dist", "average_pixel_dist_parties"]
for var in vars:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p.loc[df_p["wave"]==2], x=var, hue="treatment_wave2",
    ↪ax=axs[0], alpha=0.6, palette=cmapTreatment)
    for t in [0,1]:
        sns.regplot(df_p_bothwaves.loc[df_p_bothwaves.
    ↪wave2_treatment_wave2==t], x="wave1_"+var, y="wave2_"+var, ax=axs[1],
    ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"}, color=cmapTreatment[t])
        axs[1].set_aspect("equal")
        print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

display(df_p[vars].describe())
```



```
correlation wave 2 wave 1 average_pixel_dist: 0.544919637973932
correlation wave 2 wave 1 average_pixel_dist_parties: 0.5180154828198414
```

	average_pixel_dist	average_pixel_dist_parties
count	2131.000000	2131.000000
mean	0.281627	0.320045
std	0.073028	0.089704
min	0.057501	0.068942
25%	0.230760	0.254271
50%	0.280162	0.320929
75%	0.331161	0.387069
max	0.549351	0.577399

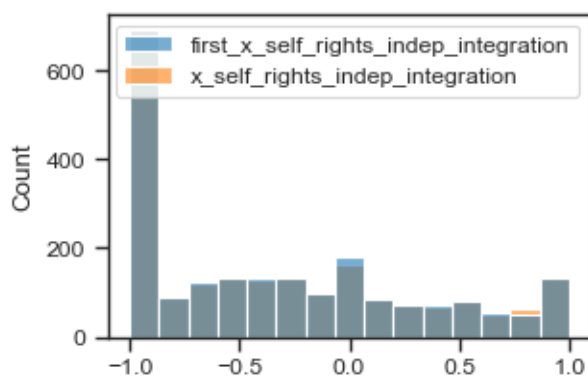
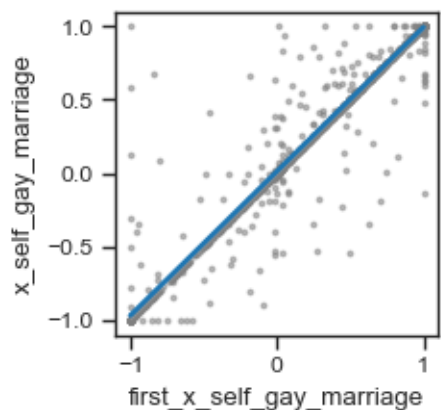
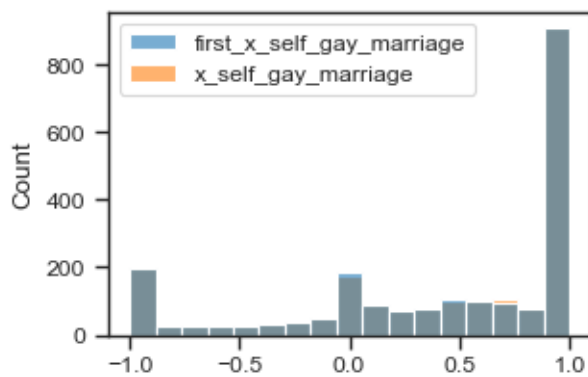
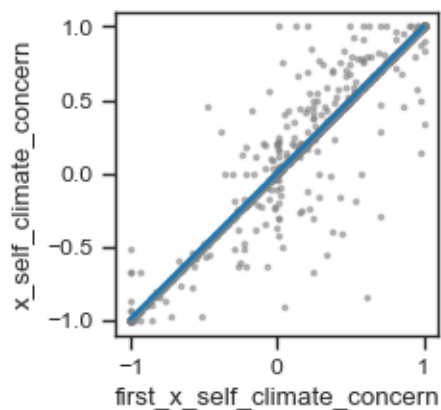
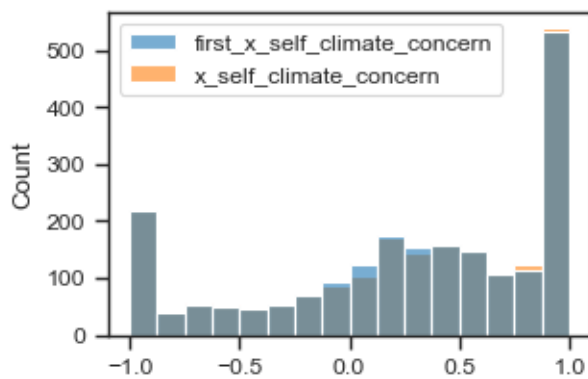


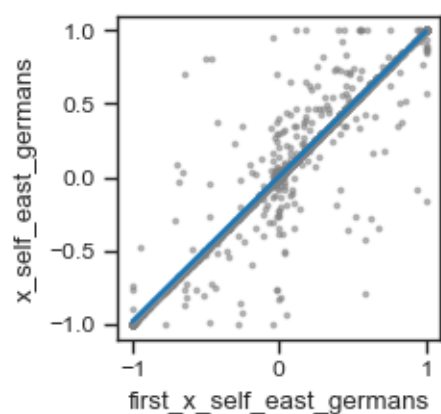
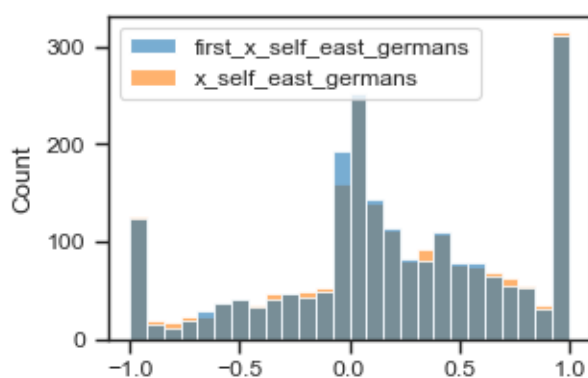
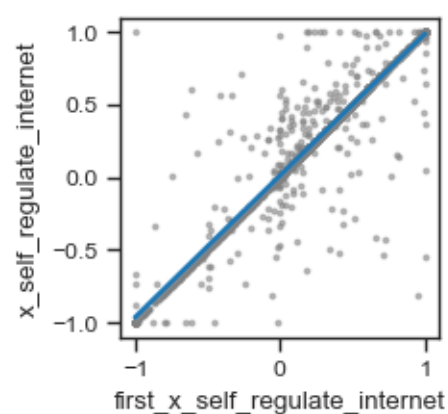
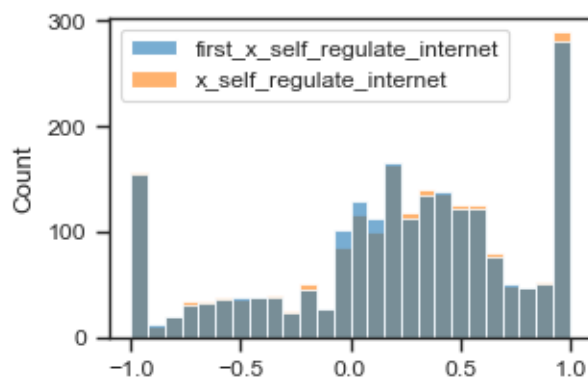
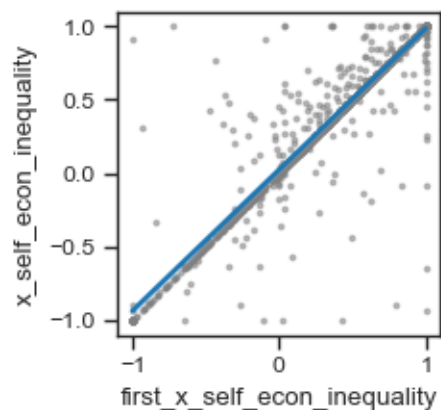
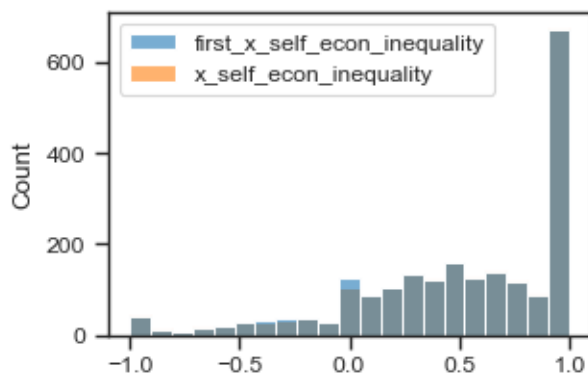
[]:

5 Opinions

```
[54]: # %%  
# Changes in opinion within the survey wave on slide 1 and slide 5(?)  
  
vars = [f"x_self_{q}" for q in questions_sc]  
varsPrior = [f"first_x_self_{q}" for q in questions_sc]  
wave = 2  
for var, varp, q in zip(vars, varsPrior, questions_sc):  
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))  
    sns.histplot(df_p[[varp, var]], ax=axs[0], alpha=0.6)  
    sns.regplot(df_p, x=varp, y=var, ax=axs[1], scatter_kws={"s":3, "alpha":0.  
↪5, "color":"grey"})  
    axs[1].set_aspect("equal")  
    print(f"correlation wave {wave} {var} and {varp}: {df_p[[varp, var]].corr().  
↪values[1,0]}")  
    ax.set_title(q)  
    fig.tight_layout()  
  
print(f"number of people who changed their opinions: {dict(zip(questions_sc, ↪  
↪((np.abs(df_p[[var for var in vars]].values - df_p[[varp for varp in ↪  
↪varsPrior]].values)>0).sum(axis=0))))}")
```

```
correlation wave 2 x_self_climate_concern and first_x_self_climate_concern:  
0.9859844235430527  
correlation wave 2 x_self_gay_marriage and first_x_self_gay_marriage:  
0.9813192325633138  
correlation wave 2 x_self_rights_indep_integration and  
first_x_self_rights_indep_integration: 0.9728781866327584  
correlation wave 2 x_self_econ_inequality and first_x_self_econ_inequality:  
0.9550145082719753  
correlation wave 2 x_self_regulate_internet and first_x_self_regulate_internet:  
0.9649260664845698  
correlation wave 2 x_self_east_germans and first_x_self_east_germans:  
0.9702255933872661  
number of people who changed their opinions: {'climate_concern': np.int64(222),  
'gay_marriage': np.int64(195), 'rights_indep_integration': np.int64(214),  
'econ_inequality': np.int64(229), 'regulate_internet': np.int64(260),  
'east_germans': np.int64(270)}
```



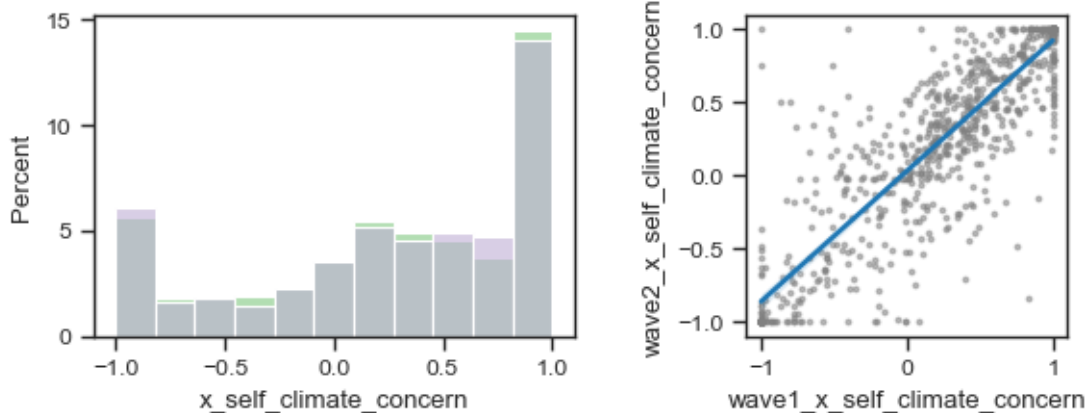


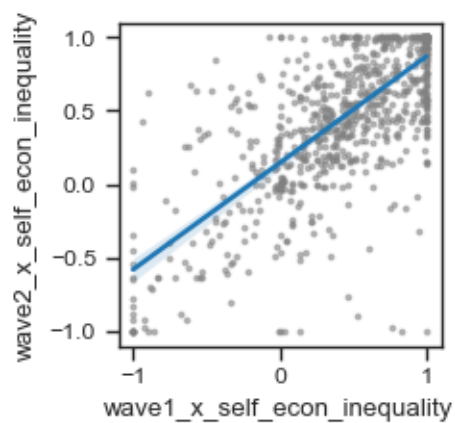
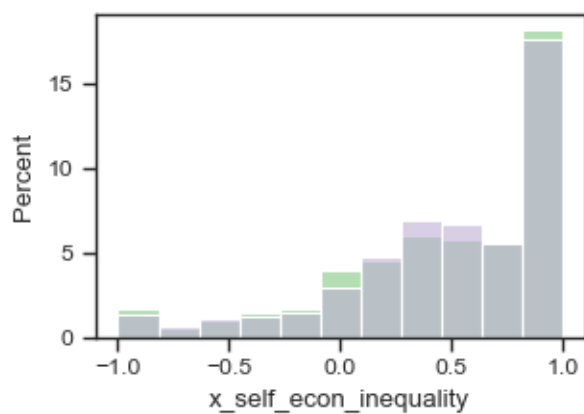
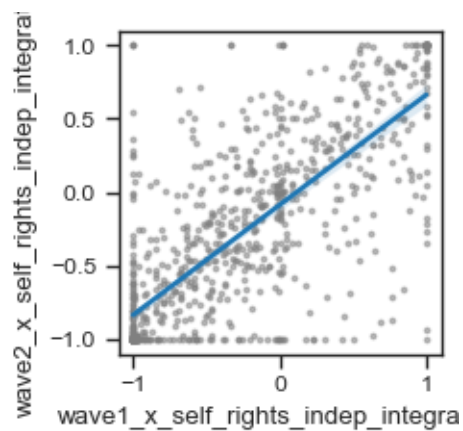
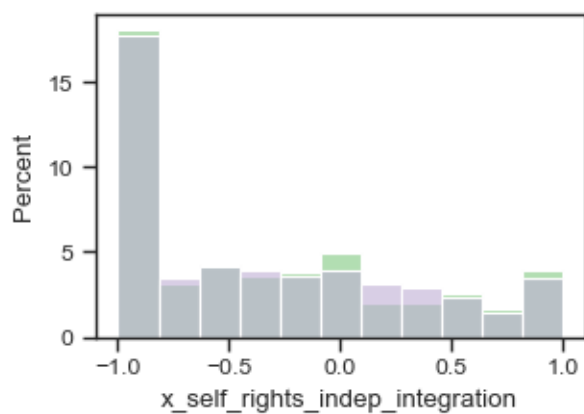
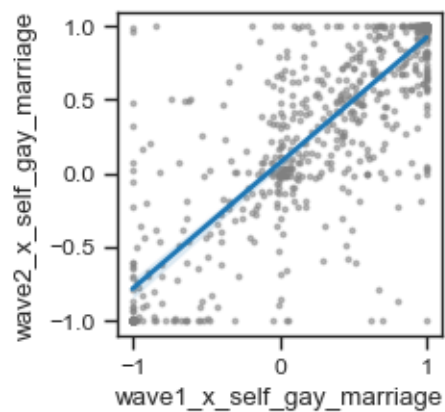
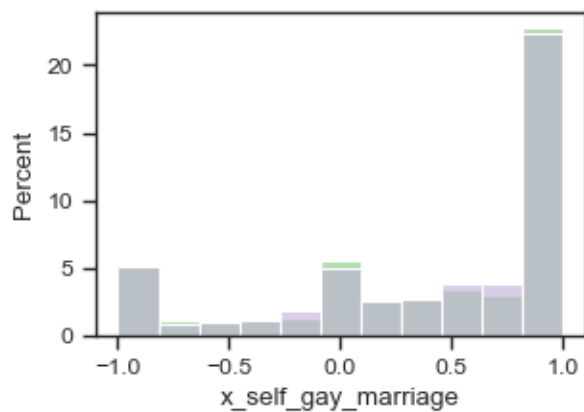
5.0.1 opinion distributions

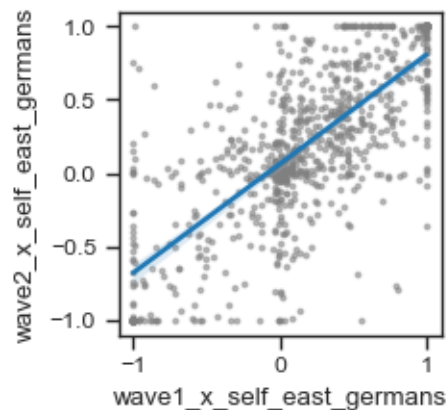
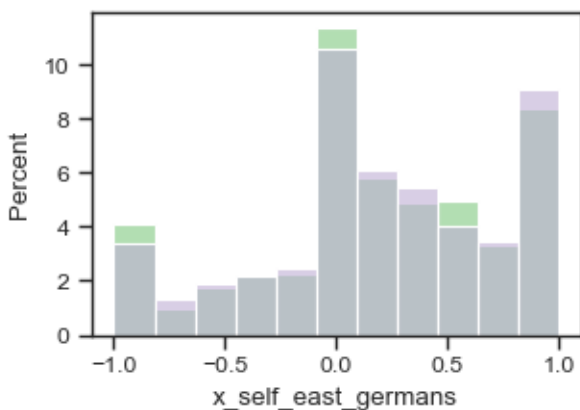
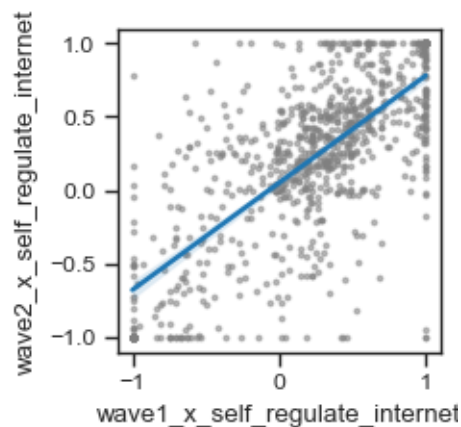
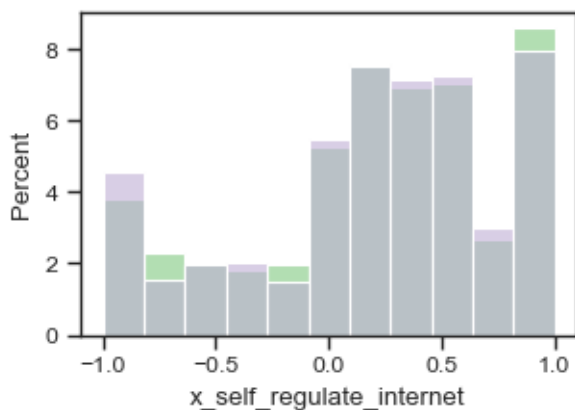
```
[69]: # %%
vars = [f"x_self_{q}" for q in questions_sc]
for var, q in zip(vars, questions_sc):
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p.loc[df_p.id.isin(df_p_bothwaves.id)], x =
    ↪f"x_self_{q}",ax=axs[0], hue="wave", bins=11, binrange=(-1,1), alpha=0.6,
    ↪legend=False, stat="percent", multiple="layer", kde=False, palette=cmapWave,
    ↪hue_order=[2,1])
    sns.regplot(df_p_bothwaves[[f"wave{w}_"+var for w in [1,2]]],
    ↪x="wave1_"+var, y="wave2_"+var, ax=axs[1], scatter_kws={"s":3, "alpha":0.5,
    ↪"color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

# display(df_p[[var for var in vars]].describe())
```

```
correlation wave 2 wave 1 x_self_climate_concern: 0.8879664029577369
correlation wave 2 wave 1 x_self_gay_marriage: 0.8589363352551868
correlation wave 2 wave 1 x_self_rights_indep_integration: 0.7655027089300674
correlation wave 2 wave 1 x_self_econ_inequality: 0.7500067135841224
correlation wave 2 wave 1 x_self_regulate_internet: 0.7286592697676602
correlation wave 2 wave 1 x_self_east_germans: 0.7475457548236663
```







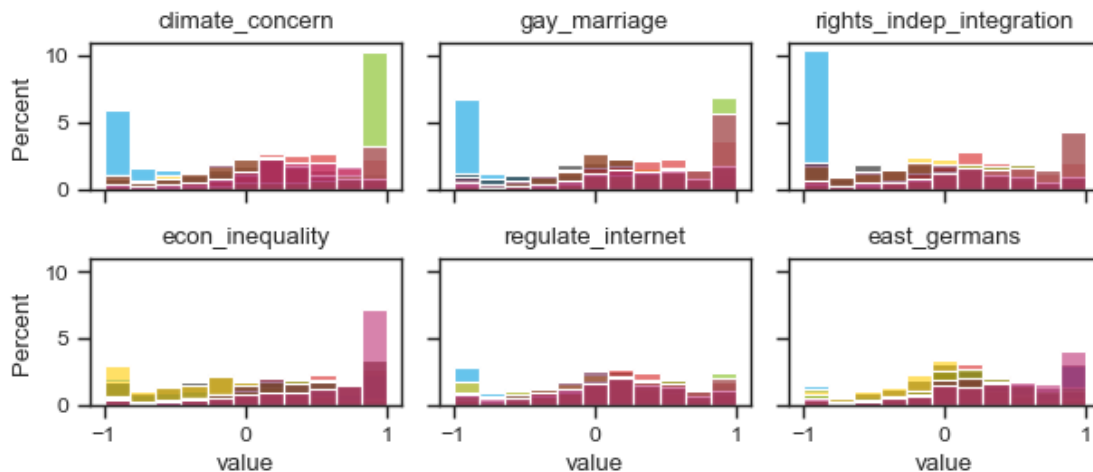
```
[70]: # %%
fig, axs = plt.subplots(2,3, figsize=(12/2.54,7/2.54), sharex=True, sharey=True)
for ax, q in zip(axs.flatten(), questions_sc):
    sns.histplot(df_p, x = f"x_self_{q}", hue="wave", bins=11, binrange=(-1,1),
    ↪ax=ax, alpha=0.6, legend=False, stat="percent", multiple="layer",
    ↪kde=False, palette=cmapWave, hue_order=[2,1])
    ax.set_title(q)
fig.suptitle("Own opinions")
fig.tight_layout()
```

Own opinions

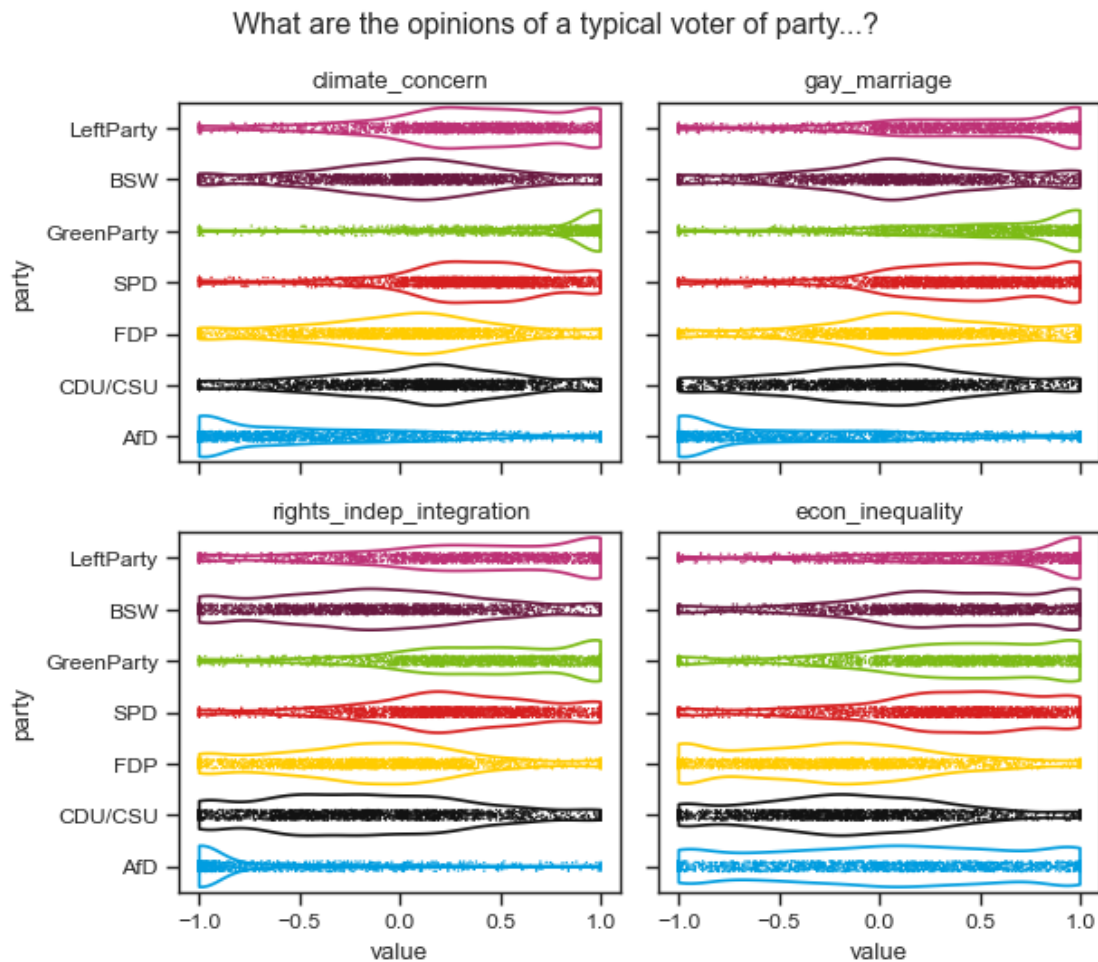


```
[71]: # %%
fig, axs = plt.subplots(2,3, figsize=(16/2.54,8/2.54), sharex=True, sharey=True)
for ax, q in zip(axs.flatten(), questions_sc):
    a = df_p[[f"x_{p}_{q}" for p in partiesVars]].melt(var_name="party", )
    a["party"] = a["party"].apply(lambda x: x.split("_")[1])
    sns.histplot(a, x="value", hue="party", palette=party_cmap,
    ↪ hue_order=partiesVars, bins=11, binrange=(-1,1), ax=ax, alpha=0.6,
    ↪ legend=False, stat="percent", multiple="layer", kde=False)
    ax.set_title(q)
fig.suptitle("What are the opinions of a typical voter of party...?")
fig.tight_layout()
```

What are the opinions of a typical voter of party...?

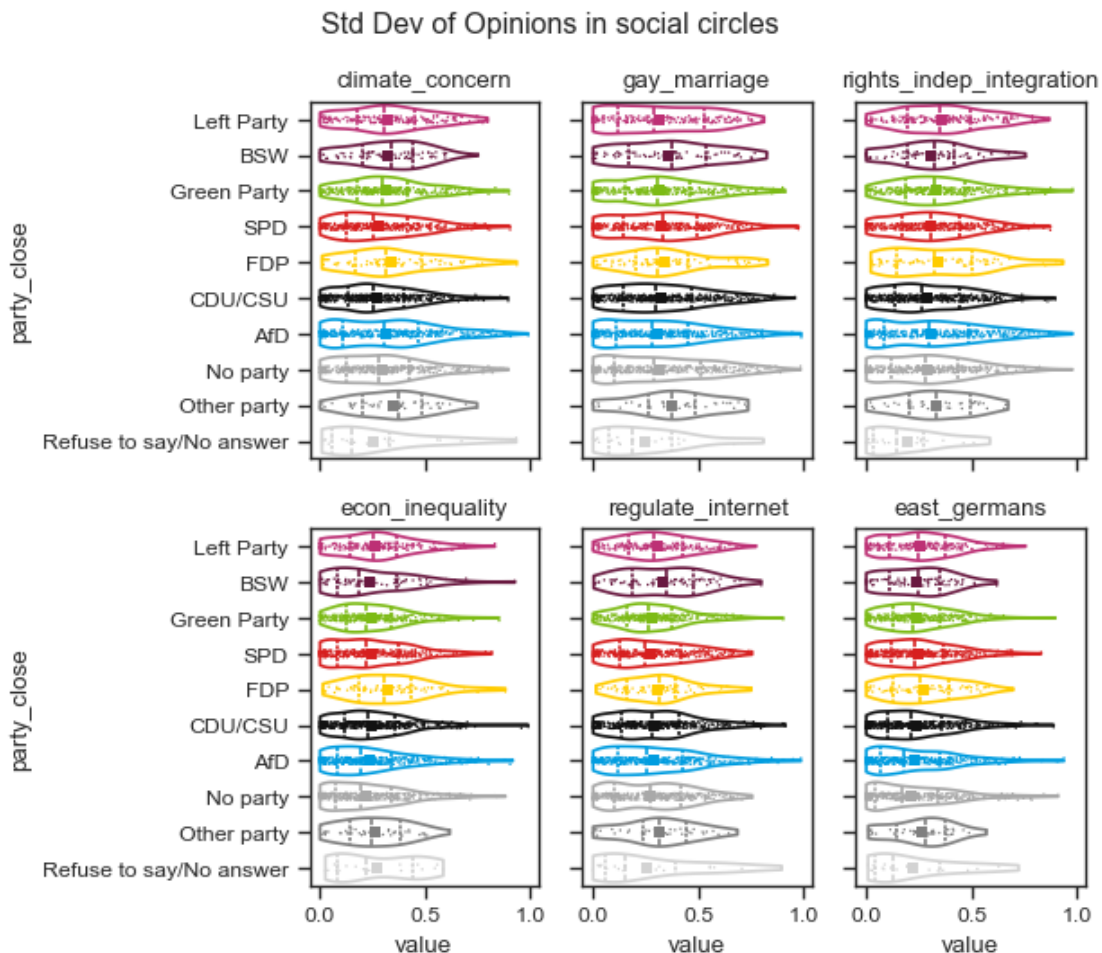



```
[72]: # %%
fig, axs = plt.subplots(2,2, figsize=(16/2.54,14/2.54), sharex=True,
    ↪sharey=True)
fig.suptitle("What are the opinions of a typical voter of party...?")
for ax, q in zip(axs.flatten(), questions_sc):
    a = df_p[[f"x_{p}_{q}" for p in partiesVars]].melt(var_name="party")
    a["party"] = a["party"].apply(lambda x: x.split("_")[1])
    sns.violinplot(a, x="value", y="party", hue="party", fill=False,
    ↪inner=None, palette=party_cmap, hue_order=partiesVars, ax=ax, legend=False,
    ↪cut=0, )
    sns.stripplot(a, x="value", y="party", hue="party", palette=party_cmap,
    ↪hue_order=partiesVars, ax=ax, legend=False, size=1)
    ax.set_title(q)
fig.tight_layout()
```



6 Std Dev of Opinions in social circles

```
[73]: # %%
fig, axs = plt.subplots(2,3, figsize=(16/2.54,14/2.54), sharex=True,
    ↳sharey=True)
fig.suptitle("Std Dev of Opinions in social circles")
for ax, q in zip(axs.flatten(), questions_sc):
    a = df_p[[f"std_socialCircle_ops_{q}"]+[f"party_close"]].
    ↳melt(var_name="social_circle", id_vars=[f"party_close"])
    sns.violinplot(a, x="value", y=f"party_close", hue=f"party_close",
    ↳fill=False, palette=party_cmap, hue_order=parties_full, ax=ax,
    ↳legend=False, cut=0, inner="quart", order=parties_full)
    sns.stripplot(a, x="value", y=f"party_close", hue=f"party_close",
    ↳palette=party_cmap, hue_order=parties_full, ax=ax, legend=False, size=1)
    sns.stripplot(a.groupby(f"party_close")["value"].mean().reset_index(),
    ↳x="value", y=f"party_close", hue=f"party_close", palette=party_cmap,
    ↳hue_order=parties_full, ax=ax, legend=False, size=5, marker="s")
    ax.set_title(q)
fig.tight_layout()
```



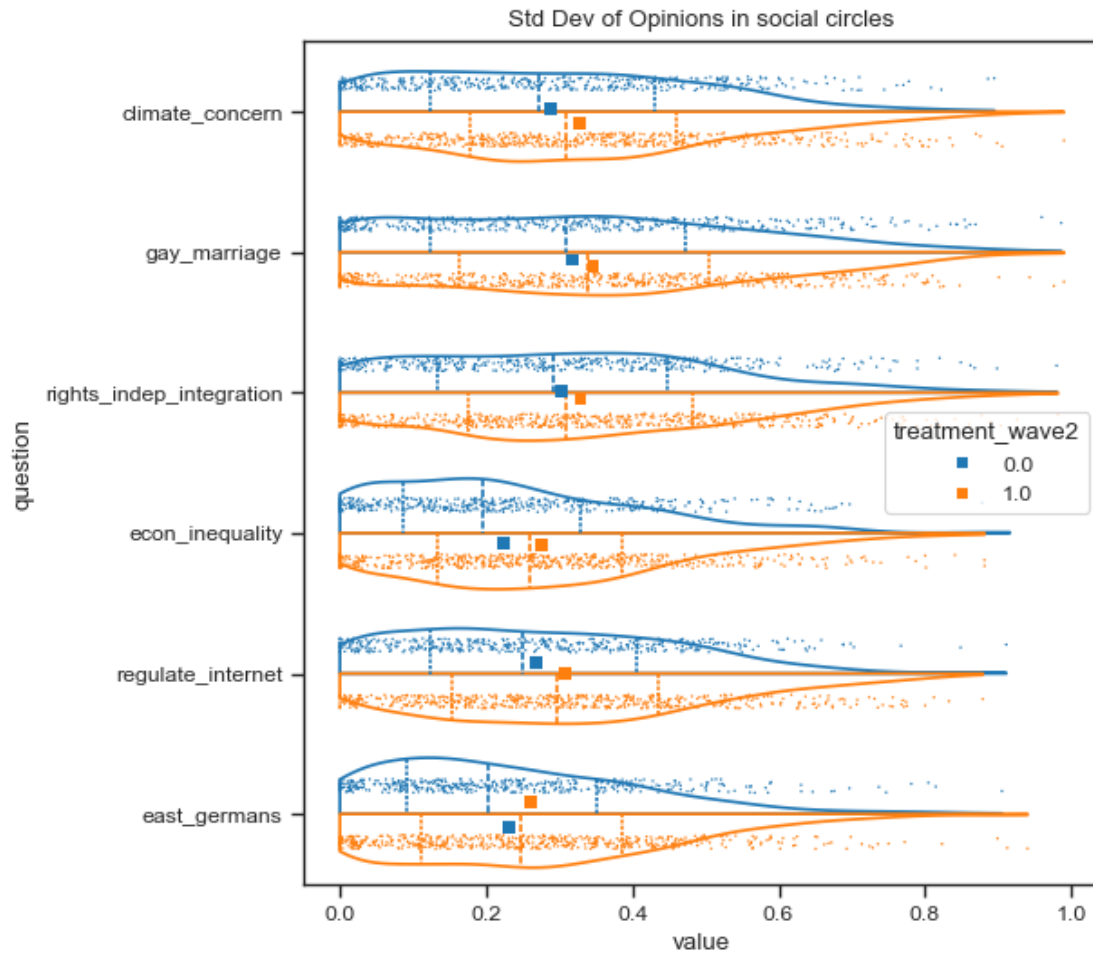
6.1 Treatment

```
[74]: # %%  
print(f"Treatment: {df_p['treatment_wave2'].value_counts().to_dict()}")
```

Treatment: {1.0: 539, 0.0: 422}

```
[75]: # %%  
fig, ax = plt.subplots(1,1, figsize=(16/2.54,14/2.54))  
ax.set_title("Std Dev of Opinions in social circles")  
a = df_p[[f"std_socialCircle_ops_{q}" for q in  
    ↪ questions_sc]+["treatment_wave2"]].melt(var_name="question",  
    ↪ id_vars=[f"treatment_wave2"])  
a["question"] = a["question"].apply(lambda x: "_".join(x.split("_")[3:]))  
a = a.dropna()  
print(len(a))  
sns.violinplot(a, x="value", y="question", split=True, hue="treatment_wave2",  
    ↪ fill=False, ax=ax, legend=False, cut=0, inner="quart")  
sns.stripplot(a, x="value", y="question", hue=f"treatment_wave2", ax=ax,  
    ↪ legend=False, size=1, dodge=True)  
sns.stripplot(a.groupby([f"treatment_wave2", "question"])["value"].mean().  
    ↪ reset_index(), x="value", y=f"question", hue=f"treatment_wave2", ax=ax,  
    ↪ legend=True, size=5, marker="s")  
fig.tight_layout()
```

5766



```
[76]: # %%
fig, axs = plt.subplots(2,3, sharex=True, sharey=True)
w=2
fig.suptitle("ALTERNATIVE: Std Dev of Opinions in social circles")

for ax, qq in zip(axs.flatten(), questions_sc):
    sns.boxplot(df_p, x="treatment_wave2", y=f"std_socialCircle_ops_{qq}",
    ↪ax=ax, fill=False, whis=[5,95], fliersize=0)
    sns.stripplot(df_p, x="treatment_wave2", y=f"std_socialCircle_ops_{qq}",
    ↪ax=ax, size=1,alpha=0.3)
    sns.stripplot(df_p.groupby("treatment_wave2")[f"std_socialCircle_ops_{qq}"].
    ↪mean().reset_index(), x="treatment_wave2", y=f"std_socialCircle_ops_{qq}",
    ↪ax=ax, size=10, marker="s",alpha=0.4)
    ax.set_title(qq)
    ax.set_xlabel("")
    ax.set_ylabel("social circle opinion std")
```

```

axs[-1,1].set_xlabel("Treatment")
fig.tight_layout()

```

ALTERNATIVE: Std Dev of Opinions in social circles

